
Desarrollo de una plataforma de simulación para
el entrenamiento y análisis de estrategias en
Aprendizaje Federado.

Development of a simulation framework for the
training and analysis of Federated Learning
strategies.



Trabajo de Fin de Grado
Curso 2025–2026

Autor

Sergio Jiménez Cubas

Director

Antonio Sarasa Cabezuelo

José Eduardo Raimundo Fernando

Grado en Ingeniería Informática

Facultad de Informática

Universidad Complutense de Madrid

Desarrollo de una plataforma de
simulación para el entrenamiento y análisis
de estrategias en Aprendizaje Federado.
Development of a simulation framework
for the training and analysis of Federated
Learning strategies.

Trabajo de Fin de Grado en Ingeniería Informática

Autor

Sergio Jiménez Cubas

Director

Antonio Sarasa Cabezuelo

José Eduardo Raimundo Fernando

Convocatoria: *Junio 2026*

Calificación: *9.5*

Grado en Ingeniería Informática

Facultad de Informática

Universidad Complutense de Madrid

9 de junio de 2026

Dedicatoria

*A mi familia por aguantarme toda la vida.
A mis amigos por acompañarme en este
camino y en los que quedan.
Y a Laura por ser mi apoyo fundamental estos
años.*

Agradecimientos

En primer lugar, quiero agradecer a mis tutores, Antonio Sarasa y José Raimundo, por el tiempo dedicado a la supervisión de este trabajo y por hacer posible el desarrollo y la presentación de este proyecto.

También quiero agradecer a mi familia, por darme las oportunidades para poder llegar hasta aquí y apoyarme en todo momento; a mis amigos de toda la vida, que siempre están ahí para amenizar los momentos difíciles, y a Laura, por aguantarme en los momentos de bajón y compartir conmigo todo este camino día a día.

Gracias.

Resumen

Desarrollo de una plataforma de simulación para el entrenamiento y análisis de estrategias en Aprendizaje Federado.

FedBench es una aplicación de escritorio desarrollada para la simulación de redes de aprendizaje federado. Este es un paradigma del aprendizaje automático que permite el entrenamiento de modelos de inteligencia artificial de manera distribuida, preservando la privacidad de los datos. El objetivo principal de la herramienta es permitir la simulación de un entrenamiento distribuido y la generación de informes con sus métricas. Además, proporciona al usuario una interfaz intuitiva que le permite gestionar sus proyectos, integrar sus propios *datasets* y modelos, y realizar predicciones sobre redes entrenadas.

Con el objetivo de validar la aplicación, se ha realizado un experimento comparando cada una de las estrategias de agregación desarrolladas. Los resultados muestran que la aplicación permite entrenar los modelos de forma efectiva y analizar su rendimiento bajo las diferentes configuraciones.

Palabras clave

FedBench, Aprendizaje Federado, Inteligencia Artificial, Data Privacy, Aggregation Strategies, Performance Metrics.

Abstract

Development of a simulation framework for the training and analysis of Federated Learning strategies.

FedBench is a desktop application developed for federated training simulation. This is a machine learning paradigm that enables distributed training of models, ensuring data privacy. The main objective of the tool is to facilitate distributed training simulation and report generation using the resulting metrics. Furthermore, it provides an intuitive interface that enables users to manage their projects, integrate their own datasets and models, and execute predictions using trained models.

To validate the application, an experiment was conducted to evaluate and compare the different aggregation strategies implemented in the project. The results show that the application enables efficient model training and performance analysis using different configurations.

Keywords

FedBench, Federated Learning, Artificial Intelligence, Privacidad de Datos, Estrategias de Agregación, Análisis de Métricas

Índice

1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	1
1.3. Plan de trabajo	2
1.3.1. Diagrama de Gantt	3
Introduction	5
1.4. Motivation	5
1.5. Goals	5
1.6. Work plan	6
1.6.1. Gantt chart	7
2. Estado de la Cuestión	9
2.1. Inteligencia Artificial	9
2.2. Aprendizaje Federado	10
2.2.1. Optimización	10
2.2.2. Estrategias de agregación	11
2.2.3. Aplicaciones	13
3. Tecnologías Utilizadas	15
4. Arquitectura y Modelo de Datos	17
4.1. Arquitectura	17
4.2. Modelo de datos	18
5. Implementación	21
5.1. Gestión de Usuarios	21
5.2. Gestión de nodos	26
5.3. Gestión de proyectos	27
6. Experimentos	43
7. Conclusiones y Trabajo Futuro	51

7.1. Conclusiones	51
7.2. Trabajo Futuro	51
7.2.1. Funcionalidades futuras	51
Conclusions and Future Work	55
7.3. Conclusions	55
7.4. Future work	55
7.4.1. Future features	55
Bibliografía	59
A. Casos de uso	61
A.1. Actores	61
A.2. Casos de uso	62
B. Manual de Instalación	81
C. Manual de Usuario	83
C.1. Autenticación	83
C.2. Acciones del usuario administrador	85
C.3. Usuario registrado	87
C.4. Gestión de cuenta	95
D. Ejemplos de informe	97
D.1. Informe de regresión	98
D.2. Informe de clasificación	101
Índice alfabético	105

Índice de figuras

1.1. Diagrama de Gantt del proyecto	4
1.2. Project Gantt chart	8
4.1. Arquitectura de la aplicación	17
4.2. Arquitectura de la aplicación	19
5.1. Implementación de Inicio de sesión	22
5.2. Verificación de contraseña	22
5.3. Crear Cuenta	23
5.4. Función de hashing	23
5.5. Caso de Uso de Registro de Usuario	24
5.6. Pantalla de perfil	24
5.7. Caso de uso de Actualización de Usuario	25
5.8. Pantalla de perfil en gestión de usuarios	25
5.9. Trigger activado al eliminar un proyecto	26
5.10. Caso de uso de Eliminar usuario	26
5.11. Creación de un nodo	26
5.12. Constructor de la clase <i>Node</i>	27
5.13. Eliminación de un nodo	27
5.14. Página principal	28
5.15. Ejemplo de formulario cumplimentado	28
5.16. Cargar features de un proyecto	29
5.17. Creación de un proyecto	29
5.18. Copiar modelo a la dirección del proyecto	29
5.19. Eliminación de un proyecto	30
5.20. Implementación de las sugerencias en la búsqueda	30
5.21. Modificar un proyecto	31
5.22. Caso de uso Añadir un <i>dataset</i>	32
5.23. Validación de un <i>dataset</i>	32
5.24. Cálculo de <i>path</i> del último <i>dataset</i>	33
5.25. Entrenar un proyecto	34
5.26. Preparación de un entrenamiento	35

5.27. Bucle de entrenamiento	36
5.28. Cálculo de la corrección	36
5.29. Recopilación de métricas por ronda.	37
5.30. Realizar una predicción	37
5.31. Código de la predicción.	38
5.32. Confirmar una predicción	39
5.33. Cambios en <i>dataset</i> por nodo	40
5.34. Pantalla de métricas de un proyecto	40
5.35. Tabla de precisión, <i>recall</i> y <i>f1_score</i> por nodo y por clase	41
5.36. Cálculo de precisión, <i>recall</i> y <i>f1_score</i>	41
5.37. Distribución de residuos	42
5.38. Cálculo del gráfico de dispersión	42
6.1. Transformación de .json a .csv apto para la aplicación	44
6.2. Parámetros de los experimentos	45
6.3. Gráficas resultantes del entrenamiento con FedAvg en el dataset Synthetic.	46
6.4. Gráficas resultantes del entrenamiento con FedMed en el dataset Synthetic.	46
6.5. Gráficas resultantes del entrenamiento con SCAFFOLD en el dataset Synthetic.	47
6.6. Gráficas resultantes del entrenamiento con SSFed en el dataset Synthetic.	48
6.7. Comparativa de la precisión global en porcentaje para cada algoritmo.	48
C.1. Manual: Inicio de Sesión	83
C.2. Manual: Crear Cuenta	84
C.3. Manual: Introducir usuario para recuperar contraseña	84
C.4. Manual: Introducir frase de recuperación y contraseña nueva	85
C.5. Manual: Lista de nodos	85
C.6. Manual: Lista de usuarios	86
C.7. Manual: Perfil de un usuario registrado	87
C.8. Manual: Lista de proyectos	87
C.9. Manual: Modelo base	88
C.10. Manual: Creación de un proyecto nuevo.	89
C.11. Manual: Buscar proyecto	89
C.12. Manual: Configuración de un proyecto.	90
C.13. Manual: Añadir un dataset.	91
C.14. Manual: Pantalla de preparación de entrenamiento.	91
C.15. Manual: Realizar una predicción.	92
C.16. Manual: Lista de proyectos con confirmaciones pendientes.	92
C.17. Manual: Pantalla de configuración de un proyecto con confirmaciones pendientes.	93
C.18. Manual: Confirmar predicción.	93

C.19.Manual: Métricas de un proyecto.	94
C.20.Manual: Guardar un informe.	94
C.21.Manual: Ver perfil.	95
C.22.Manual: Modificar perfil.	96

Índice de tablas

6.1. Distribución de clases Non-IID por cada nodo del experimento Synthetic.	47
A.1. Caso de Uso de Login	62
A.2. Caso de Uso de Logout	63
A.3. Caso de Uso de Modificar perfil	64
A.4. Caso de Uso de Baja de usuario	65
A.5. Caso de Uso de Registro de usuario	66
A.6. Caso de Uso de Recuperar contraseña	67
A.7. Caso de Uso de Alta de nodo	68
A.8. Caso de Uso de Baja de nodo	69
A.9. Caso de Uso de Creación de un proyecto	70
A.10.Caso de Uso de Configuración de un proyecto	71
A.11.Caso de Uso de Realización de predicción	72
A.12.Caso de Uso de Añadir <i>dataset</i>	73
A.13.Caso de Uso de Confirmación de resultados de un proyecto	75
A.14.Caso de Uso de Descarga de informe del proyecto	76
A.15.Caso de Uso de Eliminación de un proyecto	77
A.16.Caso de Uso de Buscar proyecto	78
A.17.Caso de Uso de Actualización de modelos en proyectos	79
A.18.Caso de Uso de Visualización de resultados y métricas del proyecto .	80

Introducción

“Individually, we are one drop. Together, we are an ocean.”
— Ryunosuke Satoro

1.1. Motivación

La inteligencia artificial se ha abierto paso en múltiples sectores, desde el desarrollo de software hasta la generación de contenido multimedia, convirtiéndose en una herramienta fundamental en el día a día, pero dependiente de muchos datos. Esto plantea distintos desafíos de gran importancia, principalmente en cuanto a la gestión de los datos personales y la privacidad. Con el modelo de aprendizaje tradicional el usuario corre el riesgo de perder el control de sus datos, enviándolos a servidores externos, que luego utilizarán los datos para entrenar y que en ese proceso pueden ser interceptados para ser utilizados de forma fraudulenta, entre otros posibles riesgos. La cantidad de datos recolectados diariamente no solo supone un peligro para la privacidad del usuario, sino que provoca un cuello de botella debido al gran volumen de información a transferir. Con el objetivo de solucionar estos dos problemas, surge el aprendizaje federado, que se basa en el almacenamiento y entrenamiento de los modelos de forma distribuida. Al procesar los datos de los usuarios localmente y transmitir las actualizaciones de los pesos del modelo al servidor, se garantiza que no salgan datos sensibles del origen. Esto es especialmente relevante en diversos sectores en los que se trabaja con datos sensibles, como puede ser el de la medicina, en el que la confidencialidad es imprescindible.

Con objeto de adentrarse en esta rama del *machine learning*, se pretende crear una herramienta que permita experimentar, visualizar y comparar los mecanismos específicos del aprendizaje federado de forma sencilla y accesible.

1.2. Objetivos

Objetivo general: El objetivo principal de este proyecto es proporcionar una aplicación que permita al usuario simular rondas de entrenamiento de aprendizaje federado para resolver diferentes problemas y generar informes con los que pueda

evaluar el rendimiento del aprendizaje.

Objetivos específicos:

1. **Especificación de requisitos:** Realización de un documento que especifique los requisitos de la aplicación.
2. **Recopilación de información:** Obtención de información sobre el aprendizaje federado y búsqueda de documentación sobre los distintos *frameworks* para simular un entrenamiento de ese estilo.
3. **Desarrollo del módulo gestión de usuarios:** Implementación de la interfaz y lógica correspondiente al módulo de gestión de usuarios.
4. **Desarrollo del módulo de gestión de nodos:** Implementación de la interfaz y la lógica correspondiente al módulo de gestión de nodos.
5. **Desarrollo del módulo de gestión de proyectos:** Implementación de la interfaz y la lógica correspondiente al módulo de gestión de proyectos, incluyendo entrenamiento y generación de informes.
6. **Experimentos y resultados:** Realización de experimentos dentro de la aplicación, utilizando distintos parámetros para un mismo problema y comparación de los resultados.

1.3. Plan de trabajo

Utilizando como base los objetivos planteados en el punto anterior, se distinguen las siguientes fases:

- **Fase 1: Especificación de requisitos:** Captura de los requisitos de la aplicación y realización de un documento en el que se especifican. Se han diferenciado las siguientes secciones:
 - Gestión de Usuarios
 - Gestión de Nodos
 - Gestión de Proyectos
- **Fase 2: Recopilación de información:** Investigación sobre el aprendizaje federado, distintas funciones de agregación existentes, comparación del rendimiento entre las distintas funciones, comparación del rendimiento entre el aprendizaje federado y el aprendizaje centralizado. Además, búsqueda de documentación de bibliotecas y *frameworks* para desplegar o simular redes de aprendizaje federado.
- **Fase 3: Desarrollo del módulo de gestión de usuarios:** Implementación de la interfaz y la lógica de los casos de uso relacionados con la gestión de usuarios. En esta fase se implementan inicio de sesión, alta y baja de un usuario, recuperación de contraseña, visualización y modificación de un usuario.

Además, en esta primera fase del desarrollo de la aplicación, se inicializa la base de datos para permitir la persistencia de estas operaciones.

- **Fase 4: Desarrollo del módulo de gestión de nodos:** Implementación de la interfaz y la lógica de la gestión de nodos, que incluye alta y baja de nodos. También se prepara el entorno para almacenar los ficheros de los *datasets*, necesarios para el entrenamiento.
- **Fase 5: Desarrollo del módulo de gestión de proyectos:** Implementación de la interfaz y lógica de los casos de uso pertenecientes a la gestión de proyectos:
 - Alta y baja de proyecto, configuración de proyecto, adición de un *dataset* al nodo de un proyecto, confirmación del resultado de una predicción y búsqueda de proyecto.
 - Lógica, tanto del entrenamiento, como de la realización de predicciones y la carga dinámica de modelos.
 - Cálculo de métricas y generación de informes estructurados en base a las métricas calculadas de los entrenamientos.
- **Fase 6: Experimentos y resultados:** Definición de unas bases para realizar distintos experimentos con el objetivo de comprobar el rendimiento del entrenamiento en la aplicación y la comparación entre las distintas funciones de agregación implementadas en el proyecto. Documentación de los experimentos y análisis de los resultados obtenidos.
- **Fase 7: Memoria del proyecto:** Realización de la memoria del proyecto.

1.3.1. Diagrama de Gantt

A continuación, se muestra un diagrama de Gantt con la planificación realizada:

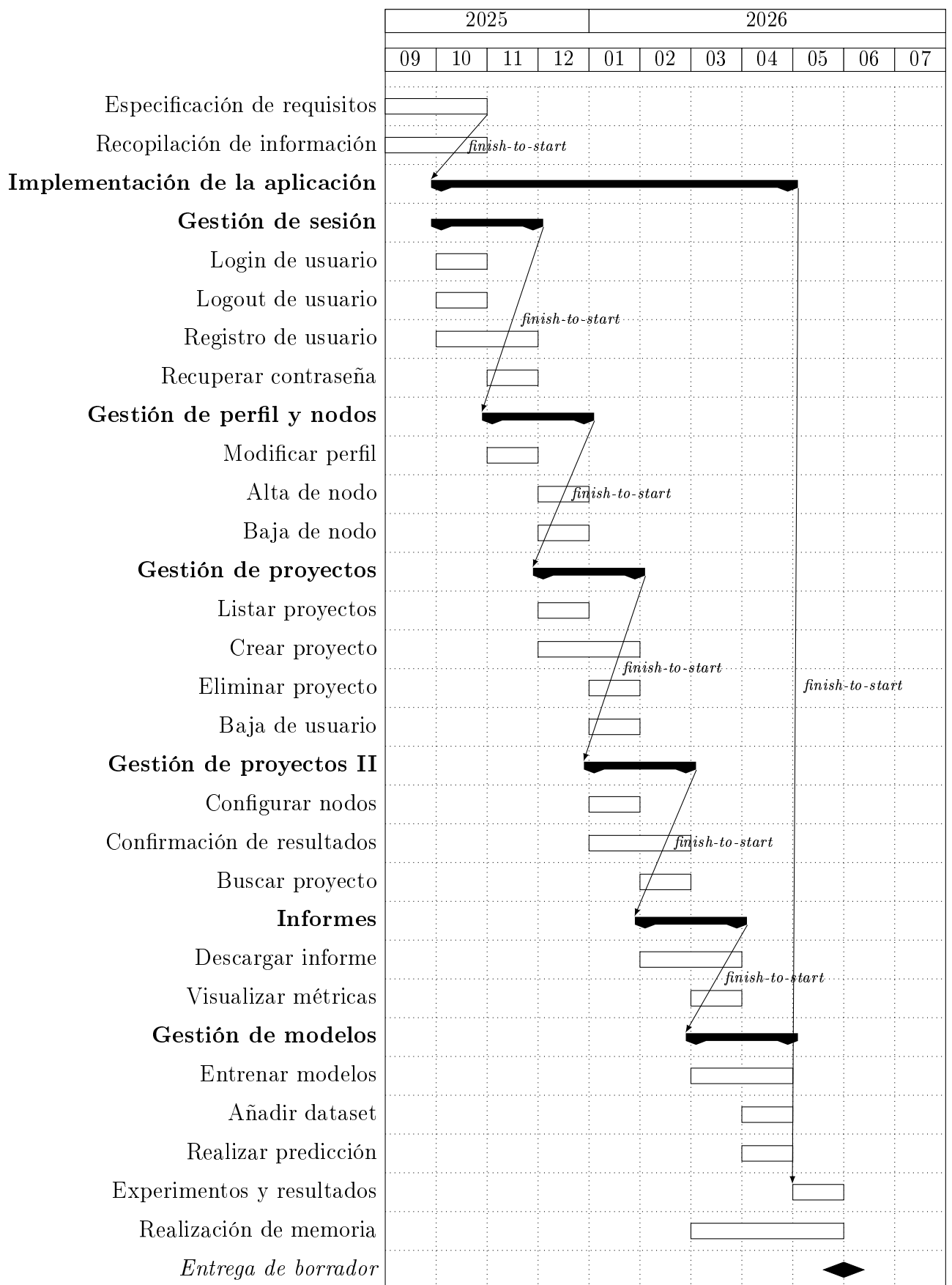


Figura 1.1: Diagrama de Gantt del proyecto

Introduction

1.4. Motivation

AI has made its way into multiple sectors, from software development to multimedia content creation, becoming a fundamental tool in everyday life, but relying on large amounts of data. This presents a number of significant challenges, mainly in terms of personal data management and privacy. In centralized training, users risk losing control over their personal data by sending them to an external server, where these data will later be used to train the model. During this process, they can be exposed and misused, among other risks. The amount of data collected on a daily basis is not only a threat to users' privacy, but it also causes a bottleneck due to the high bandwidth required for transfer. Federated learning was developed to solve these problems. It is based on distributed storage and training. Since data are processed locally, and the device only transmits the weights updates to the server, sensitive data is guaranteed to remain on the device. This is specially relevant in some sectors where sensitive data are involved, such as the medical field, in which confidentiality is an essential requirement.

To further investigate this branch of machine learning, this project aims to create a tool that allows users to experiment, visualise and compare the specific mechanisms of federated learning in an intuitive and accesible way.

1.5. Goals

General goal: This project's main goal is to develop an application that allows users to simulate federated learning training rounds to address various tasks and generate reports to evaluate model performance.

Specific goals:

1. **Requirements specification:** Definition of a document specifying the application requirements.
2. **Information research:** Gathering theoretical and practical foundations of federated learning and searching for documentation on different federated learning simulation frameworks.

3. **Development of the user management module:** Implementation of the interface and logic for the user management module.
4. **Development of the node management module:** Implementation of the interface and logic for the node management module.
5. **Development of the project management module:** Implementation of the interface and logic for the project management module, including training and report generation.
6. **Experiments and results:** Conducting experiments using the app, adjusting parameters for the same problem and comparing results.

1.6. Work plan

Using the objectives described in the previous section as a base, we can distinguish the following phases:

- **Phase 1: Requirements specification:** Documentation of the application requirements and producing a document specifying them. The following sections have been identified:
 - User Management
 - Node Management
 - Project Management
- **Phase 2: Information research:** Investigation into federated learning and various existing aggregation methods, including a performance comparison between them, as well as between distributed and centralised learning. Additionally, research into libraries and frameworks documentation for deploying or simulating federated learning networks.
- **Phase 3: Development of the user management module:** Implementation of the interface and logic for user management use cases. In this phase, the following are implemented: user login, user registration and deactivation, password recovery, and the ability to view and edit user profiles. Furthermore, in this first development phase, the database is initialised to enable the persistence of these operations.
- **Phase 4: Development of the node management module:** Implementation of the interface and logic for node management use cases, including node creation and deletion. Additionally, the environment is configured to host and manage the dataset files required for training.
- **Phase 5: Development of the project management module:** Implementation of the interface and logic for the use cases relating to project management:

- Project registration and deletion, project configuration, adding a dataset to a project node, confirming the result of a prediction, and searching for a project.
 - Logic, both in terms of training and in making predictions and dynamically loading models.
 - Calculation of metrics and generation of structured reports based on the metrics calculated from the training sessions.
- **Phase 6: Experiments and results:** Establishing a framework for conducting various experiments with the aim of assessing the effectiveness of the training in practice and comparing the different aggregation functions implemented in the project. Documenting the experiments and analysing the results obtained.
- **Phase 7: Project report:** Preparation of the project report.

1.6.1. Gantt chart

The following Gantt chart shows the planning carried out:

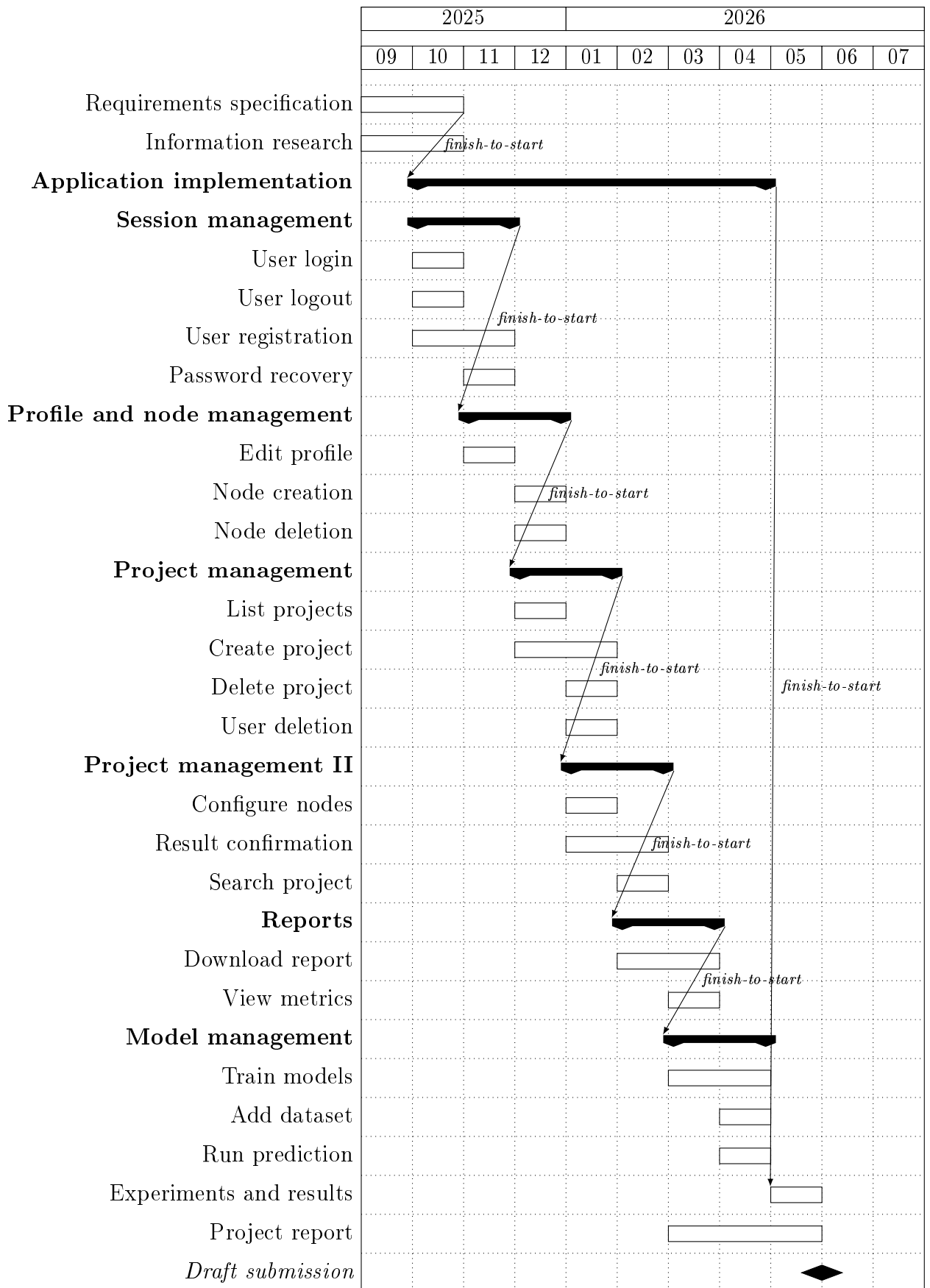


Figure 1.2: Project Gantt chart

Estado de la Cuestión

2.1. Inteligencia Artificial

Se puede definir la inteligencia artificial (IA) como el conjunto de tecnologías capaces de simular en cierta medida la inteligencia humana. Entre las habilidades que simulan, se encuentran el aprendizaje, la toma de decisiones, la creatividad y la autonomía. El aprendizaje automático es el paradigma predominante. Este consiste en el desarrollo de algoritmos capaces de identificar patrones significativos en grandes conjuntos de datos, con el objetivo de construir modelos capaces de realizar predicciones o tomar decisiones ante información nueva sin necesidad de haberlo programado específicamente para ello. Para conseguir este objetivo de forma efectiva, es crucial contar con una muestra de datos lo suficientemente representativa; de lo contrario, el modelo corre el riesgo de incurrir en sobreajuste (overfitting), un fenómeno donde el algoritmo memoriza los datos de entrenamiento en lugar de aprender el patrón real, perjudicando al resultado de las predicciones con información nueva. Muchos de estos datos utilizados para el entrenamiento tienen su origen en datos recopilados por los dispositivos de los usuarios.

Hoy en día, la mayoría de los usuarios de Internet utilizan un teléfono móvil (International Telecommunication Union, 2023). Estos dispositivos son capaces de capturar grandes cantidades de datos con gran precisión y, al ser dispositivos de uso habitual y móviles, muchos de los datos capturados son de carácter privado. El uso de estos datos para el aprendizaje automático tradicional presenta dos problemas: la vulneración de la privacidad y el elevado ancho de banda necesario para centralizar los datos. Con el objetivo de superar estos desafíos, McMahan et al. (2023) propone el **Aprendizaje Federado**, un paradigma que permite el entrenamiento de modelos de forma distribuida.

2.2. Aprendizaje Federado

El aprendizaje federado es un paradigma del aprendizaje automático que propone un entrenamiento coordinado por el servidor, pero realizado en los clientes. Cada cliente entrena con su propio *dataset* y envía al servidor las actualizaciones en los pesos de su red, que agrega las de todos los clientes utilizando alguna estrategia de agregación. A este proceso se le denomina **ronda**. Comparando el rendimiento de los distintos algoritmos de agregación, Nilsson et al. (2018) compara varios entre ellos y también con un modelo de aprendizaje centralizado. La conclusión final es que este último es representa el límite superior de precisión, solamente equiparado en este aspecto por el algoritmo *FedAvg* en problemas **IID** (independientes e idénticamente distribuidos), que tiene un rendimiento similar. Por ello, la elección de este paradigma no puede venir dada por la precisión del entrenamiento, sino por la reducción de costes para movilizar y almacenar los datos, y la reducción de responsabilidades y peligros en el ámbito de la privacidad.

2.2.1. Optimización

En el aprendizaje centralizado, el principal cuello de botella es la capacidad de procesamiento del dispositivo. En cambio, en aprendizaje federado, el mayor problema es el coste de la comunicación. Esto es debido a que, al repartir el entrenamiento entre una cantidad considerable de dispositivos, la cantidad de procesamiento realizado por cada uno se reduce drásticamente y sin embargo, cada uno de ellos tiene que enviar las actualizaciones al servidor.

Según McMahan et al. (2023), hay dos formas de reducir estos costes: utilizar mayor número de clientes trabajando en cada ronda y añadir computación adicional en cada dispositivo para reducir las rondas de comunicación necesarias para entrenar. Esto se puede hacer utilizando un cálculo más complejo que el cálculo básico de gradiente. Según sus resultados que, tras alcanzar un mínimo de paralelismo, el mayor *speedup* se consigue añadiendo más computación en cada cliente, permitiendo que cada cliente se comunique menos veces con el servidor antes de converger.

Además, menciona que hay varias propiedades que diferencian la optimización federada a la de otros típicos problemas de optimización distribuida:

- **Non-IID**: El entrenamiento de un cliente se realiza con sus datos, que con toda probabilidad no son representativos del conjunto de clientes.
- **Desbalanceado**: Hay clientes que aportarán muchos más datos que otros.
- **Masivamente distribuidos**: Esperan que el número de clientes sea considerablemente mayor que el número de ejemplos medio por cliente.
- **Comunicación limitada**: Muchos dispositivos no estarán conectados de manera habitual.

Estas propiedades provocan que los algoritmos de optimización tradicionales no sean directamente aplicables. La comunicación intermitente de los clientes y el carácter *Non-IID* de los datos obligan a que el proceso de agregación sea capaz de

conseguir un modelo global que no tenga un sesgo hacia los clientes con mayor volumen de información o mayor disponibilidad de red.

Por esto, es de vital importancia el diseño de estrategias de agregación para cumplir con esa condición y que el uso del aprendizaje federado sea viable en condiciones reales.

2.2.2. Estrategias de agregación

A continuación, se explican las distintas estrategias de agregación utilizadas durante este proyecto:

2.2.2.1. Federated Averaging (*FedAvg*)

Presentada por McMahan et al. (2023), constituye el estándar entre las estrategias, debido a su gran relación rondas/resultados. Esta funciona haciendo una media ponderada de los pesos de cada cliente basada en el número de muestras de cada cliente sobre el global de muestras utilizadas.

Aunque al ponderarlo por su participación global se reduce el riesgo de que el ruido afecte al modelo global, es posible que un cliente con un volumen de datos lo suficientemente significativo y con datos ruidosos o muy distintos de todos los demás (consecuencia de la naturaleza *Non-IID* del aprendizaje federado), puede arrastrar al modelo global hacia un error.

2.2.2.2. Federated Median (*FedMedian*)

Esta estrategia fue propuesta por Yin et al. (2021). Consiste en calcular la mediana de las actualizaciones de los clientes para cada peso del modelo.

Esta estrategia de agregación es más robusta que la media en entornos poco estables y con ruido, ya que es menos sensible a valores extremos. Sin embargo, esto mismo provoca que en condiciones más favorables, se pierda precisión al descartar mucha información de clientes buenos. Además, tiene mayor coste computacional al obligar al servidor a ordenar los valores de todos los clientes.

2.2.2.3. SCAFFOLD

Fue propuesta por Karimireddy et al. (2021) como solución a un problema de *FedAvg* ya mencionado: la sensibilidad ante la heterogeneidad de los datos locales de los clientes. SCAFFOLD utiliza variables de control para corregir las desviaciones locales de los clientes (*client-drift*). En una variable de control global (c), el servidor mantiene una estimación de la dirección que debería tomar el modelo. En una variable de control local (c_i), cada cliente almacena su desviación con la global. Cada ronda, los clientes ajustan cada paso del gradiente utilizando la diferencia entre c y su c_i local. Al terminar la ronda, el cliente envía al servidor los pesos nuevos y su variable de control local actualizada. Finalmente, el servidor promedia los pesos y actualiza la variable de control global calculando la media ponderada de las variaciones en las variables de control local realizadas por los clientes que participan en la ronda en base al número de clientes.

Gracias a la corrección de las desviaciones de los clientes, esta estrategia es mucho más robusta cuando los datos son altamente *Non-IID*. Además, al corregir estas desviaciones, se necesitan menos rondas para conseguir resultados. Sin embargo, el uso de esta estrategia requiere que el servidor tenga estado para poder guardar memoria de cada cliente para calcular la variable de control.

2.2.2.4. SSFed

La estrategia *SSFed* (*Statistical Significance Aggregation*) (Alsenani, 2025) consiste en aplicar un test de significación estadística a las actualizaciones de los clientes. Tras recibir las actualizaciones de los clientes, calcula la distribución estadística de los cambios. Después, se evalúa cada actualización con respecto a esta distribución. Si una actualización se encuentra fuera de un umbral de confianza definido, se considera que esa actualización no es significativa y la descarta o se escala para que su impacto sea menor.

Esta selección estadística hace que sea muy eficiente frente a datos ruidosos, descartando o minimizando las actualizaciones muy desviadas del resto. Sin embargo, la carga computacional sobre el servidor aumenta en gran medida, ya que debe calcular medias, varianzas y hacer el test para todos los pesos de los clientes. Además, al descartar actualizaciones que no son estadísticamente significativas, es posible descartar actualizaciones potencialmente útiles, por lo que un umbral mal ajustado puede provocar el sesgo de la mayoría.

2.2.2.5. Otras estrategias

Ahora se mencionarán algunas estrategias que son importantes pero no han sido implementadas:

- **FedProx**: Presentado por Li et al. (2020), introduce un término de proximidad al modelo previo enviado por el servidor en la función de pérdida que los clientes optimizan. Mitiga el *client-drift* forzando a que los modelos locales no se alejen mucho del global.
- **FedNova**: Presentado por Wang et al. (2020), pretende resolver el sesgo provocado por la diferencia de trabajo realizado por cada cliente, provocado por la diferencia de hardware o de cantidad de datos. Para ello, se normaliza la actualización dividiéndola entre el número de pasos realizados. Finalmente, el servidor las agrega, escalándolas en función de número total de pasos realizados por los clientes.
- **FedOpt**: Presentado por Reddi et al. (2021), se proponen versiones federadas de optimizadores adaptativos, como *Adagrad* o *Adam*, con el objetivo de resolver la compleja configuración y lenta convergencia de los métodos federados estándar.

2.2.3. Aplicaciones

A continuación, se describen algunas aplicaciones del aprendizaje federado, tanto soluciones ya implementadas, como estudios que demuestran su viabilidad en sectores críticos:

2.2.3.1. GBoard

Una aplicación del aprendizaje federado se encuentra en la predicción de texto para dispositivos móviles, como se detalla en Hard et al. (2019). En él, se explica cómo Google implementó el aprendizaje federado para mejorar las sugerencias de su teclado GBoard. Este se encarga de predecir la siguiente palabra más probable en una frase. Como se explica en el artículo, para entrenarlo, se ha utilizado la estrategia *FedAvg*. El modelo se entrena de forma local únicamente cuando el dispositivo está inactivo. Este modelo protege la privacidad de los usuarios, al no compartir sus mensajes con el servidor. Además, en el estudio se observa que, utilizando aprendizaje federado, el acierto en las sugerencias a los usuarios también aumenta.

2.2.3.2. Sector médico

Este paradigma es especialmente útil en el ámbito médico, en el que los datos manejados son especialmente sensibles y privados como se estudia en Sheller et al. (2020). Esta investigación se centra en la segmentación de tumores cerebrales. En él, se utilizan tanto *datasets* públicos como datos proporcionados por varias instituciones médicas, manteniendo las procedencias originales de estos datos para mantener el carácter *Non-IID* de los datos, haciendo el experimento más realista. Además, se utilizan datos externos para comprobar la efectividad del modelo en una institución nueva que no participa en el entrenamiento inicial.

El estudio concluye que los modelos entrenados con aprendizaje federado en este ámbito alcanzan un rendimiento casi equivalente al del entrenamiento centralizado. Esto permite que las distintas instituciones alrededor del mundo colaboren entre ellas, mejorando en gran medida el modelo, sin el riesgo que supone la posibilidad de compartir datos personales de los pacientes.

Tecnologías Utilizadas

A continuación se describen las distintas tecnologías y bibliotecas relevantes utilizadas para desarrollar la aplicación:

- Lenguaje principal:** La aplicación se ha desarrollado íntegramente en Python. Esto se debe principalmente a la baja curva de aprendizaje del lenguaje y a la gran cantidad de librerías que existen en él dedicadas al *Machine Learning*, ciencia de datos, estadísticas...
- Interfaz de usuario (*Tkinter*):** Para la interfaz gráfica se ha utilizado *Tkinter* (Python Software Foundation, 2023), la biblioteca estándar de Python para aplicaciones de escritorio. Se ha elegido ya que, al integrarse de forma nativa en el lenguaje, reduce dependencias y simplifica el despliegue. A pesar de existir otras bibliotecas más potentes para la realización de interfaces, *Tkinter* cubre las distintas necesidades del proyecto (formularios, navegación básica, visualización de métricas...), y ofrece equilibrio entre la simplicidad de implementación y sus capacidades.
- Aprendizaje Automático (*PyTorch* y *scikit-learn*):** Para la definición de los modelos y su entrenamiento, se ha utilizado *PyTorch* (PyTorch Foundation, 2025). Esta elección se debe a su flexibilidad en cuanto a la arquitectura de los modelos y configuración de los entrenamientos, lo que lo hace ideal para la experimentación e investigación académica. Para extraer las distintas métricas resultantes de los entrenamientos, se ha utilizado la biblioteca *scikit-learn* (Scikit-learn Developers, 2026), que ofrece herramientas que permiten el cálculo de indicadores para problemas de regresión y clasificación con una alta eficiencia.
- Aprendizaje federado (*Flower*):** Para la simulación del aprendizaje federado, se ha utilizado el *framework Flower (Flwr)* (Flower Labs, 2026), que ofrece la orquestación de rondas y estrategias de agregación, tanto para simulación, como para despliegue real de un conjunto de nodos para el entrenamiento federado. Se han modificado las estrategias de agregación que ofrece el *framework*, y se han creado otras nuevas, de forma que se guarden los pesos del modelo global agregado al finalizar el entrenamiento.

- **Persistencia de datos (*SQLite*):** La base de datos elegida para la persistencia de datos ha sido *SQLite*, debido a su facilidad de despliegue al estar almacenada en un fichero, sin depender de un servidor (SQLite Consortium, 2025); a la naturaleza local del proyecto, por lo que no se realizan un gran volumen de accesos concurrentes; y al buen rendimiento que consigue con el volumen de datos manejado en el proyecto, con tiempos de acceso muy pequeños.

Arquitectura y Modelo de Datos

4.1. Arquitectura

La arquitectura de la aplicación se define con los siguientes módulos: *gui*, *use_cases*, *security*, *services*, *repositories*, *federated*, *dbcon*, *BBDD* y *DATA*. Se muestra de manera gráfica en la figura 4.1. Con esta división, se pretende separar la vista, lógica y persistencia, con el objetivo de facilitar el mantenimiento y la escalabilidad de la aplicación.

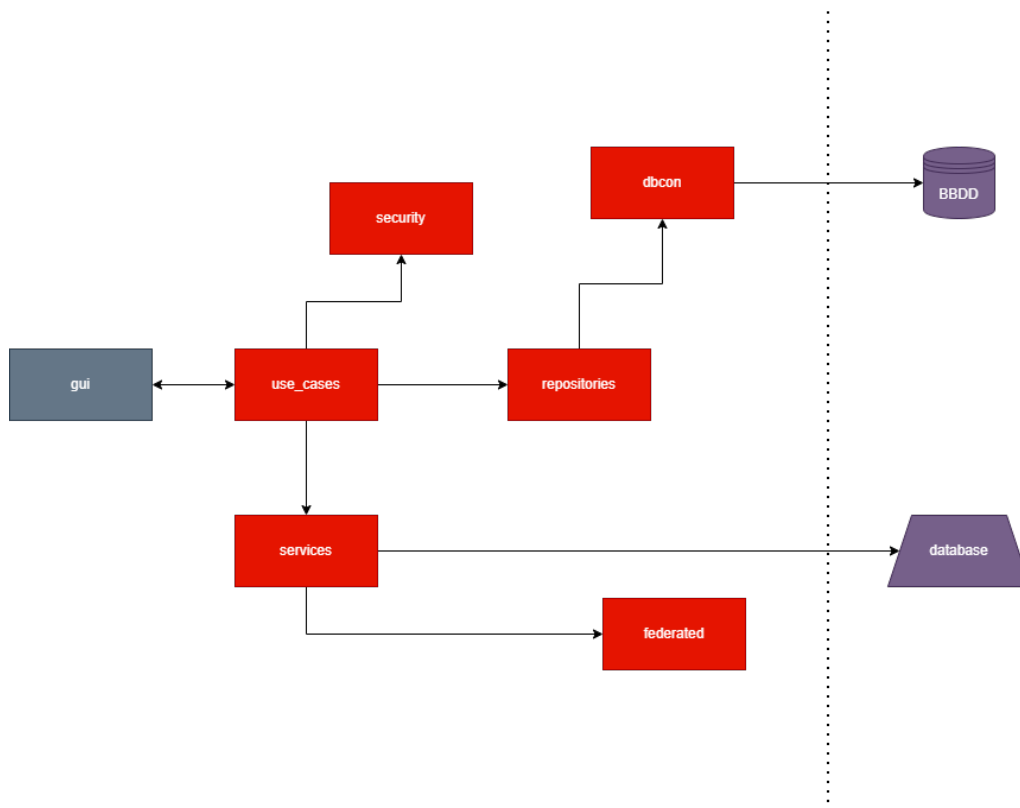


Figura 4.1: Arquitectura de la aplicación

A continuación, se describen los distintos módulos:

1. **gui:** Es la interfaz gráfica del usuario, desarrollada en *Tkinter*. Se comunica con *use_cases* para ejecutar los distintos casos de uso definidos y recibe los resultados de manera que se puedan mostrar al usuario de forma clara.
2. **use_cases:** Encapsula la lógica y estructura de los distintos casos de uso de la aplicación. Se comunica con *security* para los servicios de encriptación/autenticación, con *repositories* para obtener datos de la base de datos y con *services* para delegar la lógica de operaciones especializadas, como cálculo de métricas o acceso a *datasets*.
3. **security:** Se encarga de las tareas de encriptación, validación y autenticación, tanto de las contraseñas de los usuarios, como de sus frases de recuperación.
4. **services:** Encapsula lógica específica, que puede ser, o no, reutilizable por varios casos de uso. De esta manera, se eliminan responsabilidades de *use_cases* y se mejora la mantenibilidad.
5. **federated:** Se encarga de organizar y ejecutar todo lo relacionado con el Aprendizaje Federado. Define las estrategias de agregación, organiza los entrenamientos y realiza las predicciones.
6. **repositories:** Se compone de distintas interfaces que se comunican con *dbcon* para obtener y modificar información en la base de datos. Proporcionan las operaciones necesarias sobre las distintas tablas para completar los casos de uso correctamente.
7. **dbcon:** Gestiona la conexión con la base de datos y ejecuta comandos SQL directamente sobre ella.
8. **BBDD:** Base de datos SQLite, que contiene información de usuarios, nodos y proyectos. El único módulo que se comunica con la base de datos es *dbcon*. Está compuesta por tres tablas, como se puede ver en en la figura 4.2.
9. **database:** Directorio en el que se almacenan los artefactos utilizados para realizar los entrenamientos y predicciones. Se divide en dos:
 - **datasets:** Directorio en el que se guardan los distintos *datasets* utilizados por cada nodo, dentro de un directorio con el id del nodo en cuestión.
 - **models:** Directorio en el que se guardan los modelos utilizados por los distintos proyectos. Cada modelo se encuentra dentro de un directorio con el id del proyecto al que pertenece.

4.2. Modelo de datos

Con el objetivo de garantizar la escalabilidad de la aplicación, se ha definido un modelo de datos con una gestión de usuarios sencilla. La base de datos está formada por tres tablas:

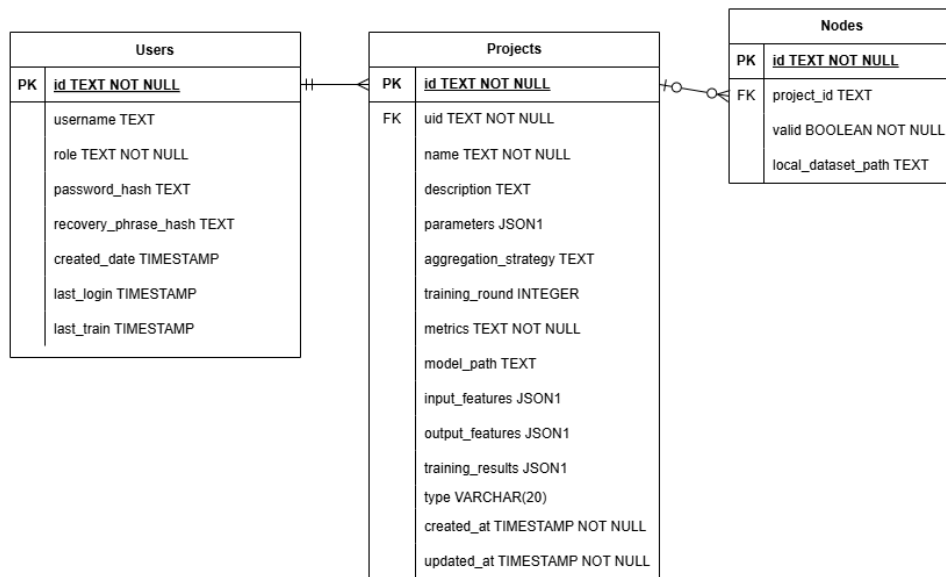


Figura 4.2: Arquitectura de la aplicación

1. **users:** Almacena la siguiente información de los usuarios registrados en la aplicación:

- **id:** Identificador único del usuario, generado cuando se registra en la aplicación. Es la clave primaria de la tabla.
- **username:** Nombre del usuario, elegido por él al registrarse, también único. Este usuario puede ser modificado posteriormente.
- **hash_password:** Contraseña cifrada del usuario.
- **hash_recoverphrase:** Frase de recuperación cifrada, definida por el usuario al registrarse. Esta frase sirve para recuperar la contraseña en caso que el usuario no la recuerde. Se opta por esta forma de verificación para la recuperación de la contraseña debido a la naturaleza estrictamente local de la aplicación. Este mecanismo garantiza el correcto funcionamiento de la aplicación en entornos aislados.
- **creation_date:** Fecha de creación/registro. Inmutable.
- **last_login:** Fecha de la última vez que el usuario inició sesión en la aplicación.
- **last_train:** Fecha del último entrenamiento realizado por el usuario con la aplicación.

2. **nodes:** Define la estructura de un nodo de entrenamiento. Almacena la siguiente información:

- **id:** Identificador único del nodo, generado al crearlo.
- **valid:** Indica si el nodo es válido para añadir a un proyecto (0=no, 1=sí).

- **project_id**: Identificador del proyecto al que pertenece (únicamente válido si `valid=0`).
 - **local_dataset_path**: Dirección en la base de datos en la que se guarda el *dataset* del nodo.
3. **projects**: Representa los proyectos activos en la aplicación, con todos sus atributos. Tiene la siguiente estructura:
- **id**: Identificador único del proyecto, generado al crearlo.
 - **uid**: Identificador del usuario dueño del proyecto (clave foránea).
 - **name**: Nombre del proyecto. Se puede modificar posteriormente.
 - **description**: Descripción del proyecto. Se puede modificar posteriormente.
 - **parameters**: Parámetros con los que se realizará el entrenamiento, en formato JSON.
 - **aggregation_strategy**: Estrategia de agregación utilizada para entrenar en el proyecto.
 - **nodes**: Lista de identificadores de los nodos pertenecientes al proyecto.
 - **unconfirmed_results**: Lista de predicciones realizadas pendientes de confirmación.
 - **training_round**: Ronda actual del entrenamiento federado.
 - **metrics**: Función de pérdida utilizada para medir el rendimiento del entrenamiento.
 - **model_path**: Ruta del modelo utilizado en el proyecto.
 - **input_features**: Nombre de los parámetros de entrada del modelo.
 - **output_features**: Nombre de los parámetros de salida del modelo.
 - **training_results**: Lista con los resultados de los entrenamientos realizados en el proyecto, ronda por ronda.
 - **type**: Tipo de problema que resuelve el proyecto (regression- problema de regresión; classification- problema de clasificación)
 - **created_at**: Fecha de creación del proyecto.
 - **updated_at**: Fecha del último entrenamiento.

Capítulo 5

Implementación

La implementación de la aplicación se ha dividido en varios módulos con el objetivo de facilitar su comprensión, mantenimiento e implementación.

A continuación se detallará cada uno de estos módulos:

5.1. Gestión de Usuarios

La gestión de usuarios permite registrar nuevos usuarios, iniciar sesión, modificar datos de la cuenta, listar usuarios y dar de baja a usuarios.

1. **Inicio de sesión:** Al acceder a la aplicación, se muestra un formulario de inicio de sesión, en el que el usuario puede introducir sus credenciales (nombre de usuario y contraseña) para obtener acceso a las funcionalidades principales de la aplicación.

Tras introducir las credenciales, estas son trasladadas al caso de uso correspondiente a la autenticación de un usuario en el módulo *use_cases*. Este comprueba si el usuario existe en base de datos. Si existe, extrae su contraseña *hasheada* y la verifica utilizando la verificación de contraseñas del módulo *security*. Una vez verificado que la combinación es correcta, se da acceso a la aplicación y se actualiza el último login del usuario en la base de datos. En las figuras 5.1 y 5.2 podemos ver la implementación de este proceso.

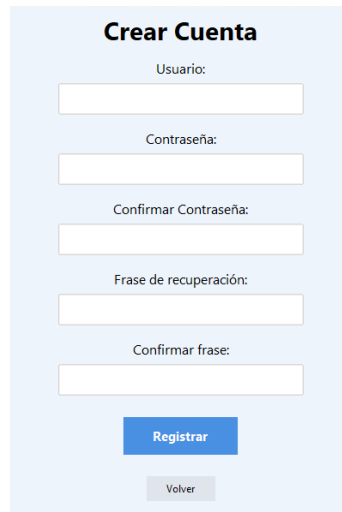
```
7 class AuthenticateUserUseCase:
8     """Authenticate one user by username and password."""
9
10    def __init__(self, user_repository: UserRepository):
11        self.user_repository = user_repository
12
13    def execute(self, username: str, password: str) -> OperationResult[dict]:
14        """Validate credentials and return user row."""
15        row = self.user_repository.get_by_username(username)
16        if not row:
17            return OperationResult(ok=False, error="Usuario o contraseña incorrectos")
18        if not verify_password(password, row.get("password_hash")):
19            return OperationResult(ok=False, error="Usuario o contraseña incorrectos")
20        updated_row = self.user_repository.update(
21            {
22                "id": row["id"],
23                "last_login": sqlite_timestamp_now(),
24            }
25        )
26        return OperationResult(ok=True, data=updated_row)
```

Figura 5.1: Implementación de Inicio de sesión

```
20 def verify_password(plain_text: str, stored_hash: str) -> bool:
21     """Validate a plain text value against a stored PBKDF2 hash."""
22     raw = (plain_text or "").strip()
23     if not raw or not stored_hash:
24         return False
25     try:
26         algorithm, rounds_text, salt_hex, digest_hex = stored_hash.split("$", 3)
27         if algorithm != "pbkdf2_sha256":
28             return False
29         rounds = int(rounds_text)
30         salt = bytes.fromhex(salt_hex)
31         expected = bytes.fromhex(digest_hex)
32     except (ValueError, TypeError):
33         return False
34     current = hashlib.pbkdf2_hmac("sha256", raw.encode("utf-8"), salt, rounds)
35     return hmac.compare_digest(current, expected)
```

Figura 5.2: Verificación de contraseña

2. **Registro de usuario:** Desde la página de Inicio de sesión, se permite registrar una nueva cuenta, pulsando el botón “No tengo cuenta”. Esto muestra la página de registro, en la que se pueden introducir los datos de la cuenta nueva. En la figura 5.3 se muestra la pantalla de registro.



Crear Cuenta

Usuario:

Contraseña:

Confirmar Contraseña:

Frase de recuperación:

Confirmar frase:

Registrar

Volver

Figura 5.3: Crear Cuenta

Cuando el usuario rellena los campos y pulsa en “Registrar”, se pasan las credenciales de la nueva cuenta al caso de uso de registro de usuario. Este comprueba que las contraseñas y las frases de recuperación coinciden y las *hashea* utilizando el algoritmo PBKDF2, como se puede ver en la figura 5.4. Después, crea el usuario en la base de datos. La implementación se muestra en la figura 5.5.

```
10 def hash_password(plain_text: str) -> str:
11     """Return a PBKDF2 hash for the provided plain text value."""
12     raw = (plain_text or "").strip()
13     if not raw:
14         raise ValueError("Password value cannot be empty.")
15     salt = secrets.token_bytes(SALT_BYTES)
16     digest = hashlib.pbkdf2_hmac("sha256", raw.encode("utf-8"), salt, PBKDF2_ITERATIONS)
17     return f"pbkdf2_sha256${PBKDF2_ITERATIONS}${salt.hex()}${digest.hex()}"
```

Figura 5.4: Función de hashing

```

13 def execute(
14     self,
15     username: str,
16     password: str,
17     password_confirm: str,
18     recovery_phrase: str,
19     recovery_confirm: str,
20 ) -> OperationResult[dict]:
21     """Create one user if validations pass."""
22     username = (username or "").strip()
23     password = (password or "").strip()
24     password_confirm = (password_confirm or "").strip()
25     recovery_phrase = (recovery_phrase or "").strip()
26     recovery_confirm = (recovery_confirm or "").strip()
27     if not username or not password or not password_confirm or not recovery_phrase or not recovery_confirm:
28         return OperationResult(ok=False, error="Rellena todos los campos")
29     if password != password_confirm:
30         return OperationResult(ok=False, error="Las contraseñas no coinciden")
31     if recovery_phrase != recovery_confirm:
32         return OperationResult(ok=False, error="Las frases de recuperación no coinciden")
33     existing = self.user_repository.get_by_username(username)
34     if existing:
35         return OperationResult(ok=False, error="El usuario ya existe")
36     user = self.user_repository.create(
37         {
38             "username": username,
39             "password_hash": hash_password(password),
40             "recovery_phrase_hash": hash_password(recovery_phrase),
41             "role": "user",
42             "creation_date": sqlite_timestamp_now(),
43         }
44     )
45     return OperationResult(ok=True, data=user)

```

Figura 5.5: Caso de Uso de Registro de Usuario

3. **Modificación de cuenta:** Desde la página de Perfil del usuario, que se puede ver en la figura 5.6, se puede pulsar en “Modificar perfil”.

Figura 5.6: Pantalla de perfil

Al pulsar el botón, se muestra el formulario de modificación de perfil, en el que se pueden cambiar los siguientes datos de la cuenta: nombre de usuario, contraseña y frase de recuperación. En caso de modificar la contraseña o frase de recuperación, es necesario escribirlas dos veces. Cuando se confirma, se manda a la implementación de caso de uso correspondiente, donde se comprueba que ambas coincidan. Si coinciden, se cifran y se actualiza la base de datos con los nuevos campos. Cabe destacar que solamente se realiza todo el proceso de actualización si hay algún dato para actualizar. Si el nombre de usuario no cambia y no se introduce ninguna contraseña ni frase de recuperación nueva, no se hace nada, como se puede ver en la implementación en la figura 5.7.

```

12 def execute(
13     self,
14     user_id: bytes,
15     username: str,
16     password: str,
17     password_confirm: str,
18     recovery_phrase: str,
19     recovery_confirm: str,
20 ) -> OperationResult[dict]:
21     """persist profile changes and return updated user row."""
22     username = (username or "").strip()
23     password = (password or "").strip()
24     password_confirm = (password_confirm or "").strip()
25     recovery_phrase = (recovery_phrase or "").strip()
26     recovery_confirm = (recovery_confirm or "").strip()
27     if not username:
28         return OperationResult(ok=False, error="El usuario no puede estar vacío")
29     current = self.user_repository.get_by_id(user_id)
30     if not current:
31         return OperationResult(ok=False, error="No se pudo cargar la cuenta actual")
32     update_payload = {"id": user_id}
33     if username != current["username"]:
34         update_payload["username"] = username
35     if password or password_confirm:
36         if password != password_confirm:
37             return OperationResult(ok=False, error="Las contraseñas no coinciden")
38         update_payload["password_hash"] = hash_password(password)
39     if recovery_phrase or recovery_confirm:
40         if recovery_phrase != recovery_confirm:
41             return OperationResult(ok=False, error="Las frases de recuperación no coinciden")
42         update_payload["recovery_phrase_hash"] = hash_password(recovery_phrase)
43     if len(update_payload) == 1:
44         return OperationResult(ok=False, error="No hay cambios para guardar")
45     updated = self.user_repository.update(update_payload)
46     return OperationResult(ok=True, data=updated)
47

```

Figura 5.7: Caso de uso de Actualización de Usuario

4. **Baja de usuario:** Desde el perfil de un usuario en la sección de gestión de usuarios (Figura 5.8), un administrador puede dar de baja a otro usuario pulsando en el botón “Eliminar usuario”.

User ID: 127548fc-6cf5-4f13-9307-f3a0aeeecd96

Nombre: sergio

Role: user

Creation date: -

Last login: 2026-05-05 15:40:59

Last train: 2026-05-05 15:40:29

Projects:

- Salary
 - Created at: 2026-05-05 15:37:24
 - Updated at: 2026-05-05 15:40:29
- Nodes
 - 6844deff-9a00-43ab-9ad0-76a5809c9fb9
 - ada69c31-d363-426c-8921-0f198c262138
 - ab9e421d-8686-4d2a-b272-3020f35591d2
 - 45dab4d0-d5ad-47bf-9d5f-1f36a493f9ec

Volver Eliminar Usuario

Figura 5.8: Pantalla de perfil en gestión de usuarios

Al pulsar y confirmar, se pasa el usuario a eliminar al caso de uso correspondiente, en el cual, primero, se eliminan los proyectos asociados al id del usuario en base de datos, lo que activa un *trigger* que pone *valid* a *false* en los nodos pertenecientes a este, como se puede ver en la figura 5.9. Después, se elimina definitivamente al usuario de la base de datos. La implementación de este caso de uso se puede ver en la figura 5.10.

```

2 CREATE TRIGGER invalidate_nodes_on_project_delete
3   before delete on projects
4   for each row
5   begin
6     update nodes set valid = false where project_id = OLD.id;
7   end;

```

Figura 5.9: Trigger activado al eliminar un proyecto

```

6 class DeleteUserUseCase:
7     """Delete one user and all associated projects."""
8
9     def __init__(
10         self,
11         user_repository: UserRepository,
12         project_repository: ProjectRepository,
13     ):
14         self.user_repository = user_repository
15         self.project_repository = project_repository
16
17     def execute(self, user_id: bytes) -> OperationResult[None]:
18         """Delete related projects first, then delete user."""
19         user = self.user_repository.get_by_id(user_id)
20         if not user:
21             return OperationResult(ok=False, error="No se encontró el usuario.")
22         projects = self.project_repository.list_by_user(user_id)
23         for project in projects:
24             self.project_repository.delete(project["id"])
25         self.user_repository.delete(user_id)
26         return OperationResult(ok=True)

```

Figura 5.10: Caso de uso de Eliminar usuario

5.2. Gestión de nodos

El módulo de gestión de nodos permite añadir y eliminar nodos que serán utilizados por los proyectos para simular los entrenamientos.

1. **Registro de un nodo:** Un usuario administrador puede añadir un nodo nuevo a la base de datos desde su página principal pulsando en el botón “Añadir un nodo”. Esto hace una petición al caso de uso correspondiente (figura 5.11) para añadirlo, el cual manda crear al nodo primero, después se le asigna un directorio para sus *datasets* en el constructor de la clase “Node” (figura 5.12) y se actualiza la base de datos con esta ruta.

```

19 def create(self) -> OperationResult[dict]:
20     """Create one node and persist computed dataset path."""
21     node_row = self.node_repository.create(
22         {"valid": 0, "project_id": "", "local_dataset_path": ""}
23     )
24     node = Node(node_row["id"], node_row["valid"], node_row["local_dataset_path"])
25     self.node_repository.update({"id": node.id, "local_dataset_path": node.local_dataset_path})
26     updated = self.node_repository.get_by_id(node.id)
27     return OperationResult(ok=True, data=updated)

```

Figura 5.11: Creación de un nodo

```
1124 class Node:
1125     def __init__(self, id, valid, project_id):
1126         self.id = id
1127         self.valid = valid
1128         self.project_id = project_id
1129         self.local_dataset_path = str(DATASETS_ROOT / f"node_{_node_uuid_str(id)}")
```

Figura 5.12: Constructor de la clase *Node*

2. **Eliminación de un nodo:** Desde la página principal, un administrador puede seleccionar un nodo y pulsar en el botón “Eliminar el nodo seleccionado”, lo que provoca una llamada a la función de eliminación del caso de uso correspondiente. Este intenta eliminar primero los *datasets* asociados al nodo. Si lo consigue, elimina al nodo de la base de datos.

```
29 def delete(self, node_id: bytes) -> OperationResult[None]:
30     """Delete one node and its local dataset by id."""
31     node_row = self.node_repository.get_by_id(node_id)
32     if not node_row:
33         return OperationResult(ok=False, error="No se encontró el nodo a eliminar.")
34
35     dataset_path = node_row.get("local_dataset_path") or ""
36     if dataset_path and os.path.exists(dataset_path):
37         try:
38             shutil.rmtree(dataset_path)
39         except OSError as exc:
40             return OperationResult(
41                 ok=False,
42                 error=f"No se pudo eliminar el dataset asociado al nodo: {exc}",
43             )
44
45     self.node_repository.delete(node_id)
46     return OperationResult(ok=True)
```

Figura 5.13: Eliminación de un nodo

5.3. Gestión de proyectos

La gestión de proyectos es la parte más importante de la aplicación. Esta permite crear, eliminar, buscar y configurar proyectos, añadir datasets a los nodos pertenecientes al proyecto, entrenar la red de un proyecto, realizar una predicción, confirmar una predicción, ver las métricas de los entrenamientos y descargar un informe PDF con los resultados de los entrenamientos.

1. **Creación de un proyecto:** Tras iniciar sesión, se muestra la pantalla principal de la aplicación, en la que se muestran los proyectos actuales y las posibles acciones a realizar, como se ve en la figura 5.14.

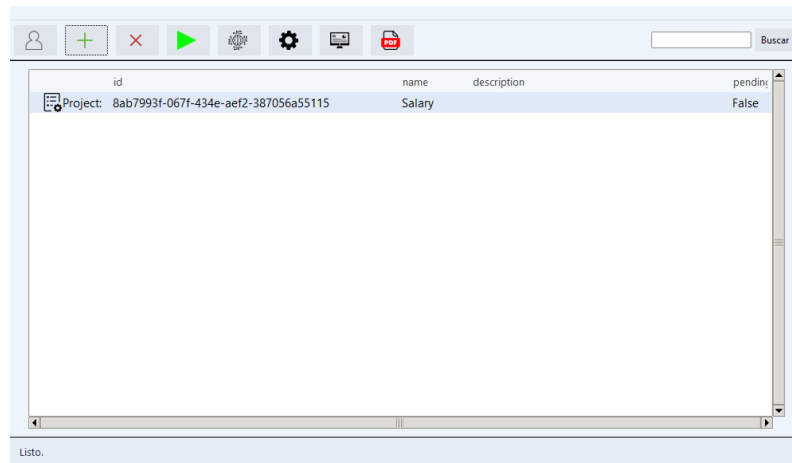


Figura 5.14: Página principal

Desde esta pantalla, el usuario puede crear un nuevo proyecto, pulsando el botón “Crear un proyecto nuevo”. Tras hacerlo, se le muestra el formulario en el que debe introducir los distintos datos del proyecto: nombre, descripción, modelo a utilizar, tipo de problema, función de pérdida, los parámetros del entrenamiento y los nodos iniciales. En la figura 5.15 se puede ver este formulario con datos de ejemplo.

The screenshot shows the project creation form. It is divided into several sections:

- Nombre:** A text input field containing "Air Quality".
- Descripción:** A text area containing "Proyecto para predecir la calidad del aire".
- Modelo:** A dropdown menu showing "air_quality_mlp_model.py" with a "Seleccionar" button next to it.
- Tipo de tarea:** A dropdown menu showing "classification".
- Función de pérdida:** A dropdown menu showing "categorical_crossentropy".
- Parámetros:** A section with several sub-sections:
 - Optimizer:** A dropdown menu showing "Adam".
 - Aggregation strategy:** A dropdown menu showing "fed_avg".
 - Epochs:** A text input field containing "3".
- Validation split:** A text input field containing "0.20".
- Batch size:** A text input field containing "32".
- Fraction fit:** A text input field containing "0.80".
- Fraction evaluate:** A text input field containing "0.50".
- Learning rate:** A text input field containing "0.0100".
- Nodos:** A list of checkboxes and IDs:
 - ☒ ada69c31-d363-425c-8921-0f198c262138
 - ☒ 45dab4d0-d5ad-47bf-9d3f-1f36a493f9ec
 - ☒ 1721617-880f-4d0d-bdeb-ee8f2b24c473
 - ☐ 35252f37-b766-44be-883d-f208d9b1a5b0
 - ☐ acc63f28-e051-46a5-9377-a0bdb3523ce5

Figura 5.15: Ejemplo de formulario cumplimentado

Cuando el usuario selecciona el modelo a utilizar, se cargan de él los parámetros de entrada y salida, además del tipo de problema, si es posible, en cuyo caso se rellena automáticamente su casilla. La implementación de este proceso se puede ver en la figura 5.16


```

75 def _load_features(self, model_class: type) -> tuple[list[Any], list[Any], str | None]:
76     """Instantiate model class and validate/get input-output features and suggested task type."""
77     model = model_class()
78     required_keys = {"input_features", "output_features"}
79     features = model.get_features()
80     is_valid = (
81         isinstance(features, dict)
82         and required_keys.issubset(features.keys())
83         and all(isinstance(features[key], list) for key in required_keys)
84     )
85     if not is_valid:
86         raise ValueError(
87             "El método get_features() debe retornar un diccionario con las claves "
88             "'input_features' y 'output_features', cuyos valores deben ser listas de nombres de características."
89         )
90     suggested = None
91     metadata = features.get("metadata")
92     if isinstance(metadata, dict):
93         suggested = self._coerce_task_type(metadata.get("type"))
94     if suggested is None:
95         suggested = self._coerce_task_type(features.get("type"))
96     return features["input_features"], features["output_features"], suggested

```

Figura 5.16: Cargar features de un proyecto

Después de rellenar los datos y pulsar “Crear”, los datos del proyecto son enviados al caso de uso de creación de proyecto, en el que se copia el modelo dentro de la carpeta del proyecto para modelos en *database*, se inserta el proyecto en la base de datos y se actualizan los nodos para indicar que pertenecen a un proyecto.

```

75 def _load_features(self, model_class: type) -> tuple[list[Any], list[Any], str | None]:
76     """Instantiate model class and validate/get input-output features and suggested task type."""
77     model = model_class()
78     required_keys = {"input_features", "output_features"}
79     features = model.get_features()
80     is_valid = (
81         isinstance(features, dict)
82         and required_keys.issubset(features.keys())
83         and all(isinstance(features[key], list) for key in required_keys)
84     )
85     if not is_valid:
86         raise ValueError(
87             "El método get_features() debe retornar un diccionario con las claves "
88             "'input_features' y 'output_features', cuyos valores deben ser listas de nombres de características."
89         )
90     suggested = None
91     metadata = features.get("metadata")
92     if isinstance(metadata, dict):
93         suggested = self._coerce_task_type(metadata.get("type"))
94     if suggested is None:
95         suggested = self._coerce_task_type(features.get("type"))
96     return features["input_features"], features["output_features"], suggested

```

Figura 5.17: Creación de un proyecto

```

75 def _load_features(self, model_class: type) -> tuple[list[Any], list[Any], str | None]:
76     """Instantiate model class and validate/get input-output features and suggested task type."""
77     model = model_class()
78     required_keys = {"input_features", "output_features"}
79     features = model.get_features()
80     is_valid = (
81         isinstance(features, dict)
82         and required_keys.issubset(features.keys())
83         and all(isinstance(features[key], list) for key in required_keys)
84     )
85     if not is_valid:
86         raise ValueError(
87             "El método get_features() debe retornar un diccionario con las claves "
88             "'input_features' y 'output_features', cuyos valores deben ser listas de nombres de características."
89         )
90     suggested = None
91     metadata = features.get("metadata")
92     if isinstance(metadata, dict):
93         suggested = self._coerce_task_type(metadata.get("type"))
94     if suggested is None:
95         suggested = self._coerce_task_type(features.get("type"))
96     return features["input_features"], features["output_features"], suggested

```

Figura 5.18: Copiar modelo a la dirección del proyecto

2. **Eliminar un proyecto:** Desde la página principal, el usuario puede seleccionar un proyecto y eliminarlo. Esto provocará que el proyecto se elimine de la base de datos, junto con sus nodos. Además, el espacio ocupado en *database* se liberará. Esto incluye *models* y *datasets* de cada nodo del proyecto.

```

19 def execute(
20     self,
21     project_id: bytes,
22     on_node_removed: Callable[[bytes], None] | None = None,
23     on_node_update_error: Callable[[Exception], None] | None = None,
24 ) -> OperationResult[None]:
25     """Delete project after invalidating nodes and removing related datasets."""
26     project_nodes = self.node_repository.list_by_project_id(project_id)
27     for node_row in project_nodes:
28         if on_node_removed is not None:
29             on_node_removed(node_row["id"])
30         try:
31             self.node_repository.update({"id": node_row["id"], "valid": 0, "project_id": ""})
32         except Exception as exc:
33             if on_node_update_error is not None:
34                 on_node_update_error(exc)
35     self.project_repository.delete(project_id)
36     return OperationResult(ok=True)

```

Figura 5.19: Eliminación de un proyecto

3. **Buscar un proyecto:** Desde la página principal, el usuario puede realizar la búsqueda de uno de sus proyectos. En la barra de búsqueda, el usuario introduce el nombre del proyecto a buscar. Mientras escribe, se muestran sugerencias de autocompletado, ordenadas en función de la distancia del coseno de los vectores resultantes de aplicar la transformación *word2vec*. Una vez se presiona la tecla “Enter” o se pulsa el botón de “Buscar”, la lista de proyectos se ordena en función de esta misma distancia.

```

584 def _get_suggestions_vector(self, search_term, max_suggestions=5):
585     names = [name.lower() for name in self.project_names]
586     suggestions = []
587     v1 = self.word2vec(search_term)
588     for word in names:
589         v2 = self.word2vec(word)
590         similarity = self.cosdis(v1, v2)
591         if word not in suggestions:
592             suggestions.append((word, similarity))
593     sorted_suggestions = sorted(suggestions, reverse = True, key=lambda suggestion: suggestion[1])
594     return [suggestion[0] for suggestion in sorted_suggestions[:max_suggestions]]

```

Figura 5.20: Implementación de las sugerencias en la búsqueda

4. **Configurar un proyecto:** Desde la página principal, el usuario puede seleccionar un proyecto, ver sus propiedades y modificar algunas de ellas pulsando en el botón “Configurar el proyecto seleccionado”. En esta página, el usuario podrá modificar el nombre, la descripción y añadir y eliminar nodos del proyecto. Una vez realizados los cambios, el usuario pulsa en “Guardar cambios”, y las propiedades del proyecto modificadas se mandan al caso de uso encargado de actualizar el proyecto. Este actualiza los datos en la base de datos

y compara los nodos previos a la modificación con los nodos nuevos. Los que estén en el primero y no en el segundo, se actualiza su casilla *valid* a cero y actualiza los que están en la lista de nodos nuevos para poner *valid* a 1 y asignar el id del proyecto en *project_id*.

```

21 def execute(
22     self,
23     project_id: bytes,
24     previous_project_row: dict[str, Any],
25     form_data: dict[str, Any],
26     on_node_removed: Callable[[bytes], None] | None = None,
27 ) -> OperationResult[dict[str, Any]]:
28     """Persist project edition and synchronize node states."""
29     update_payload = {
30         "id": project_id,
31         "uid": previous_project_row["uid"],
32         "name": form_data["name"],
33         "description": form_data["description"],
34         "parameters": json.dumps(form_data["parameters"]),
35         "aggregation_strategy": form_data["aggregation_strategy"],
36         "metrics": form_data["metrics"],
37     }
38     project_row = self.project_repository.update(update_payload)
39     old_nodes = {
40         node["id"] for node in self.node_repository.list_by_project_id(project_id)
41     }
42     new_nodes = {uuid.UUID(node_id).bytes for node_id in form_data["initial_nodes"]}
43     removed = old_nodes - new_nodes
44     for node_id in removed:
45         if on_node_removed is not None:
46             on_node_removed(node_id)
47         self.node_repository.update({"id": node_id, "valid": 0, "project_id": ""})
48     for node_id in new_nodes:
49         self.node_repository.update({"id": node_id, "valid": 1, "project_id": project_id})
50     return OperationResult(ok=True, data=project_row)

```

Figura 5.21: Modificar un proyecto

5. **Añadir *dataset* a un nodo:** Desde la página de la configuración de un proyecto, el usuario puede añadir un *dataset* a uno de los nodos del proyecto. Para ello, pulsa en “Añadir dataset”, desplegando el diálogo en el que se elige el nodo al que añadirlo y selecciona de su sistema de ficheros un fichero .csv con el *dataset* que el usuario desea agregar. Tras seleccionarlo y pulsar “Añadir”, se manda la ruta al fichero seleccionado al caso de uso correspondiente, cuya implementación se puede ver en la figura 5.22.

```

13 def execute(
14     self,
15     node_uuid: str,
16     csv_path: str,
17     in_features: list,
18     out_features: list,
19     cur_round: int,
20 ) -> OperationResult[None]:
21     """Validate source CSV and append rows to destination dataset."""
22     src = (csv_path or "").strip()
23     if not src:
24         return OperationResult(ok=False, error="Selecciona un archivo CSV.")
25     if not src.lower().endswith(".csv"):
26         return OperationResult(ok=False, error="El archivo debe tener extensión .csv.")
27     src_path = Path(src)
28     if not src_path.is_file():
29         return OperationResult(ok=False, error="No se encontró el archivo indicado.")
30     data_rows, err = self.dataset_service.validate_csv_for_project(src_path, in_features, out_features)
31     if err:
32         return OperationResult(ok=False, error=err)
33     try:
34         dest_path = self.dataset_service.get_last_dataset_path(
35             node_uuid,
36             cur_round,
37             in_features,
38             out_features,
39         )
40         self.dataset_service.append_data_rows(dest_path, data_rows or [])
41     except OSError as e:
42         return OperationResult(ok=False, error=f"No se pudo escribir el dataset: {e}")
43     return OperationResult(ok=True)

```

Figura 5.22: Caso de uso Añadir un *dataset*

Este caso de uso comprueba:

- que el fichero tenga extensión *.csv*,
- que el fichero exista,
- que la estructura del *dataset* coincida con la esperada para el proyecto. Para ello tiene que cumplir las siguientes condiciones: la primera línea debe contener los nombres de los parámetros, tanto de entrada como de salida, en el orden en que se especifica en el modelo utilizado al crear el proyecto y que no hayan filas en las que hay huecos vacíos (figura 5.23).

```

98 for row_idx, row in enumerate(rows):
99     if not any((c or "").strip() for c in row):
100         return None, f"La fila {row_idx + 1} del archivo está vacía; elimínala o rellénala."
101     while rows and not any((c or "").strip() for c in rows[-1]):
102         rows.pop()
103     if not rows:
104         return None, "El archivo no contiene ninguna fila con datos."
105     if self.labels_match_row(rows[0], expected):
106         data_rows = rows[1:]
107     if not data_rows:
108         return None, "El archivo solo contiene la cabecera; no hay filas de datos."
109     header_note = True
110     elif self.same_labels_wrong_order(rows[0], expected):
111         return None, (
112             "La cabecera contiene las mismas etiquetas que el proyecto pero en distinto orden. "
113             f"Orden requerido: {' '.join(expected)}"
114         )
115     else:
116         data_rows = rows
117         header_note = False
118     for idx, row in enumerate(data_rows, start=1):
119         if len(row) != n_cols:
120             expected_text = ", ".join(expected)
121             row_context = f"fila de datos {idx}" + (" tras la cabecera" if header_note else "")
122             return None, (
123                 f"Número de columnas incorrecto({row_context}): se esperaban {n_cols} "
124                 f"f{expected_text}), hay {len(row)}."
125             )
126     for col_idx, cell in enumerate(row):
127         if not (cell or "").strip():
128             col = expected[col_idx]
129             row_context = f"fila de datos {idx}" + (" tras la cabecera" if header_note else "")
130             return None, f"{row_context}: la columna «{col}» está vacía."
131     return data_rows, None

```

Figura 5.23: Validación de un *dataset*

Después, se calcula el path del *dataset* en el que se va a añadir el nuevo, que es aquel que coincide con el número de ronda actual. Este será utilizado en el siguiente entrenamiento. Si no existe ningún *dataset* con el número de ronda actual, se copia el último y se renombra a la última ronda. Después se añaden las filas de datos del nuevo al final del último *dataset*.

```
19 def get_last_dataset_path(  
20     self,  
21     node_uuid: str,  
22     cur_round: int,  
23     in_features: list,  
24     out_features: list,  
25 ) -> Path:  
26     """Return or create dataset file for the current round."""  
27     node_dir = DATASETS_ROOT / self.node_dir_name(node_uuid)  
28     pattern = re.compile(r"dataset_(\d+)\.csv")  
29     found_files = []  
30     if node_dir.exists():  
31         for file_path in node_dir.glob("dataset_*.csv"):  
32             match = pattern.search(file_path.name)  
33             if match:  
34                 found_files.append((int(match.group(1)), file_path))  
35     if found_files:  
36         last_dataset, file_path = max(found_files, key=lambda x: x[0])  
37         if last_dataset != cur_round:  
38             dest = node_dir / f"dataset_{cur_round}.csv"  
39             shutil.copy2(file_path, dest)  
40             file_path = dest  
41     else:  
42         file_path = self.create_new_dataset(node_dir, cur_round, in_features, out_features)  
43     return file_path
```

Figura 5.24: Cálculo de *path* del último *dataset*

6. **Entrenar la red de un proyecto:** Desde la página principal, un usuario puede simular un entrenamiento federado de un proyecto. Para ello, primero debe cumplir una condición: todos los nodos tienen que tener un *dataset*. El usuario puede iniciar el entrenamiento seleccionando el proyecto deseado y pulsando en “Iniciar entrenamiento federado”. Esto abre un diálogo en el que selecciona el número de rondas a entrenar. Tras introducir el número, pulsa “Iniciar”, lo que manda el *id* del proyecto y las rondas seleccionadas al caso de uso correspondiente.

```

32 def execute(
33     self,
34     project_id: bytes,
35     rounds: int,
36     on_progress: ProgressCallback | None = None,
37 ) -> OperationResult[dict[str, Any]]:
38     """Execute one federated training job for one project."""
39     project_row = self.project_repository.get_by_id(project_id)
40     if not project_row:
41         return OperationResult(ok=False, error="No se encontró el proyecto.")
42     nodes = self.node_repository.list_by_project_id(project_id)
43     if not nodes:
44         return OperationResult(
45             ok=False,
46             error="El proyecto no tiene nodos asignados. Añade nodos en la configuración del proyecto.",
47         )
48     missing_dataset_error = self._validate_node_datasets(project_row, nodes)
49     if missing_dataset_error:
50         return OperationResult(ok=False, error=missing_dataset_error)
51     node_ids = [str(uuid.UUID(bytes=node["id"])) for node in nodes]
52     out = self.training_service.run(
53         project_row,
54         rounds,
55         node_ids=node_ids,
56         on_progress=on_progress,
57     )
58     merged = merge_project_training_results(
59         project_row.get("training_results"),
60         out["training_results_entry"],
61     )
62     trained_at = sqlite_timestamp_now()
63     update_payload: dict[str, Any] = {"id": project_id, "training_results": merged}
64     if not project_row.get("type"):
65         update_payload["type"] = (
66             "regression"
67             if (project_row.get("metrics") or "") == "mean_squared_error"
68             else "classification"
69         )
70     cur_data_round = int(project_row.get("training_round") or 0)
71     update_payload["training_round"] = cur_data_round + int(rounds) + 1
72     update_payload["updated_at"] = trained_at
73     self.project_repository.update(update_payload)
74     self.user_repository.update(
75         {
76             "id": project_row["uid"],
77             "last_train": trained_at,
78         }
79     )
80     return OperationResult(
81         ok=True,
82         data={
83             "merged_training_results": merged,
84             "total_time_seconds": out["total_time_seconds"],
85             "project_row": project_row,
86             "rounds": rounds,
87         },
88     )

```

Figura 5.25: Entrenar un proyecto

Este valida primero que el proyecto exista y que sus nodos tengan *dataset*. Tras esto, llama al servicio de entrenamiento federado para iniciar el entrenamiento. Se configuran los parámetros del mismo, recuperándolos de la base de datos. Además, se crea la estrategia a utilizar. Las estrategias disponibles son: *FedAvg*, *FedMed*, *SCAFFOLD* y *SSFed*.

```

451 metrics = project_row.get("metrics", "categorical_crossentropy")
452 task = nm.resolve_task(params, metadata, metrics)
453
454 device = nm._get_device()
455
456 net = model_wrapper.load_model(model_path)
457 nm.load_trained_weights_into(net, project_row, model_path)
458 global_sd = {k: v.detach().cpu().clone() for k, v in net.state_dict().items()}
459 param_keys = list(global_sd.keys())
460 initial_ndarrays = _state_dict_to_ndarrays(global_sd)
461 initial_parameters = ndarrays_to_parameters(initial_ndarrays)
462
463 snapshots: list[list[np.ndarray]] = []
464 round_times: list[float] = []
465 t_start = time.perf_counter()
466
467 fraction_fit = project_row.get("fraction_fit", 1.0)
468 fraction_evaluate = project_row.get("fraction_evaluate", 1.0)
469
470 strategy = create_tracking_strategy(
471     project_row.get("aggregation_strategy"),
472     snapshots=snapshots,
473     round_times=round_times,
474     on_progress=on_progress,
475     num_rounds=num_federated_rounds,
476     t_run_start=t_start,
477     param_keys=param_keys,
478     local_epochs=int(params.get("epochs", 3)),
479     learning_rate=float(params.get("learning_rate", 0.01)),
480     total_clients=n_clients,
481     ssfed_z_threshold=float(params.get("ssfed_z_threshold", 1.96)),
482     fraction_fit=fraction_fit,
483     fraction_evaluate=fraction_evaluate,
484     min_fit_clients=int(n_clients * fraction_fit),
485     min_evaluate_clients=int(n_clients * fraction_evaluate),
486     min_available_clients=n_clients,
487     initial_parameters=initial_parameters,
488     on_fit_config_fn=lambda rnd: {"server_round": float(rnd)},
489     on_evaluate_config_fn=lambda rnd: {"server_round": float(rnd)},
490     fit_metrics_aggregation_fn=fit_metrics_noop,
491     evaluate_metrics_aggregation_fn=weighted_evaluate_metrics,
492     inplace=True,
493 )
494

```

Figura 5.26: Preparación de un entrenamiento

Después de configurar los parámetros, se preparan los clientes y se inicia el entrenamiento. La función `start_simulation(...)` del servidor de *Flwr* llama a *fit* y *evaluate* de cada cliente. La función *fit*, como se puede ver en la figura, carga los *DataLoader* para el entrenamiento, dividiéndolos en *train* y *test* en función de una semilla derivada del número de ronda. Esto hace que en ambos procesos del entrenamiento, tanto *fit* como *evaluate*, las divisiones sean las mismas. Tras cargar los datos, se llama al bucle de entrenamiento de cada cliente. El bucle de entrenamiento de los clientes se puede ver en la figura 5.27. Cuando se usa *SCAFFOLD*, además se calcula la corrección que cada cliente debe aplicar (figura 5.28).

```

519 def _train_loop(
520     net: nn.Module,
521     train_loader: DataLoader,
522     val_loader: DataLoader,
523     *,
524     epochs: int,
525     learning_rate: float,
526     optimizer_name: str,
527     criterion: nn.Module,
528     task: str,
529     metrics: str,
530     device: torch.device,
531 ) -> dict[str, Any]:
532     net = net.to(device)
533     opt = _optimizer(optimizer_name, net.parameters(), learning_rate)
534     history: dict[str, list[float]] = {"train_loss": [], "val_loss": []}
535
536     for _ in range(epochs):
537         net.train()
538         total, seen = 0.0, 0
539         for xb, yb in train_loader:
540             xb = xb.to(device)
541             yb = yb.to(device)
542             opt.zero_grad()
543             out = net(xb)
544             loss = _compute_batch_loss(out, yb, criterion, task, metrics)
545             loss.backward()
546             opt.step()
547             bs = xb.size(0)
548             total += loss.item() * bs
549             seen += bs
550         history["train_loss"].append(total / max(seen, 1))
551
552         net.eval()
553         vtot, vseen = 0.0, 0
554         with torch.no_grad():
555             for xb, yb in val_loader:
556                 xb = xb.to(device)
557                 yb = yb.to(device)
558                 out = net(xb)
559                 loss = _compute_batch_loss(out, yb, criterion, task, metrics)
560                 bs = xb.size(0)
561                 vtot += loss.item() * bs
562                 vseen += bs
563             history["val_loss"].append(vtot / max(vseen, 1))
564
565     return {"history": history}

```

Figura 5.27: Bucle de entrenamiento

```

213 out_arrays = _state_dict_to_ndarrays(net.state_dict())
214 new_c_client = []
215 for j in range(len(in_arrays)):
216     delta_x = in_arrays[j] - out_arrays[j]
217     ci_plus = c_client[j] - c_global[j] + (1 / (steps_done * lr)) * delta_x
218     new_c_client.append(ci_plus)
219
220 metrics_out["scaffold_ci_new"] = _ndarrays_to_json(new_c_client)

```

Figura 5.28: Cálculo de la corrección

Al terminar la ronda, el servidor recopila las métricas de todos los clientes por cada ronda, calculando una métrica conjunta, primero las generales y después las específicas de cada tipo de tarea. Si es clasificación, se calcula la matriz de confusión de cada nodo y los resultados de la última ronda. Si es regresión, se calculan los últimos resultados. Estos datos se guardan en la

columna *training_results* del proyecto en formato *JSON* y se utilizarán para calcular las métricas para la realización de los informes.

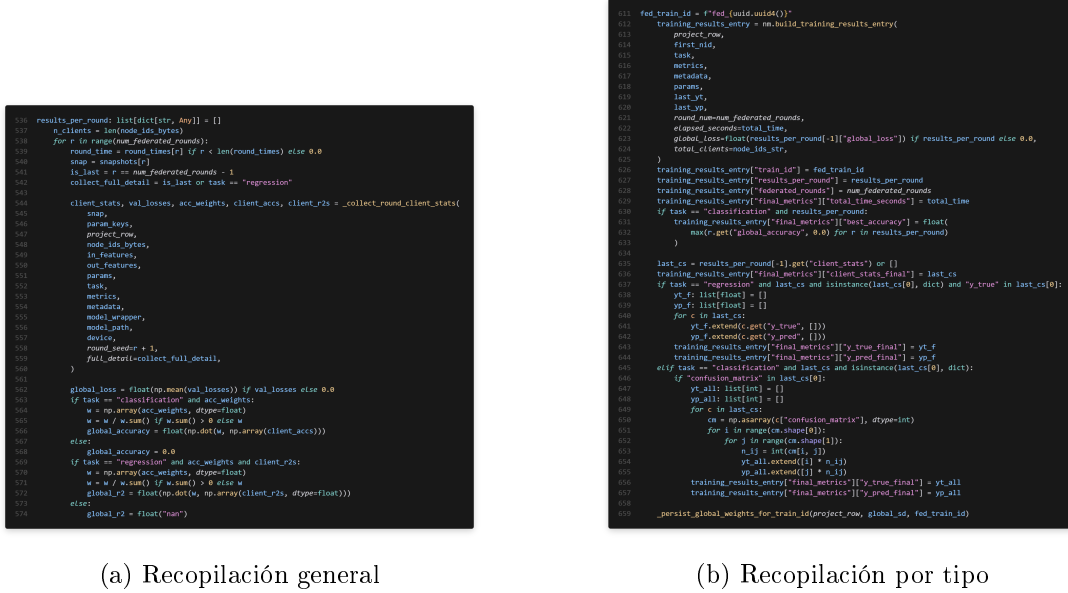


Figura 5.29: Recopilación de métricas por ronda.

- Realizar una predicción:** Desde la página principal, un usuario puede realizar una predicción en un nodo de la red de uno de sus proyectos seleccionando el botón “Realizar predicción para el proyecto seleccionado”. Esto abre un diálogo en el que el usuario puede elegir el nodo con el cual realizar la predicción y los datos de entrada, escritos en formato *CSV*. Tras rellenar estos datos y pulsar en “Predecir”, se llama al caso de uso correspondiente, con estos como argumento. Este comprueba que el nodo exista y llama al servicio de predicción.

```

24 def execute(
25     self,
26     project_id: bytes,
27     node_id: str,
28     input_values: list[float],
29 ) -> OperationResult[dict[str, Any]]:
30     """Run prediction and append pending result in project storage."""
31     project_row = self.project_repository.get_by_id(project_id)
32     if not project_row:
33         return OperationResult(ok=False, error="No se encontró el proyecto.")
34     node_row = self.node_repository.get_by_id(uuid.UUID(node_id).bytes)
35     if not node_row:
36         return OperationResult(ok=False, error="No se encontró el nodo en la base de datos.")
37     prediction_result = self.prediction_service.run_prediction(node_row, input_values, project_row)
38     pending_raw = project_row.get("unconfirmed_results") or []
39     pending = json.loads(pending_raw) if isinstance(pending_raw, str) else list(pending_raw)
40     pending.append(self._build_pending_entry(project_row, node_id, input_values, prediction_result))
41     self.project_repository.update(
42         {
43             "id": project_id,
44             "unconfirmed_results": json.dumps(pending, ensure_ascii=False),
45         }
46     )
47     return OperationResult(
48         ok=True,
49         data={"project_name": project_row["name"], "prediction_result": prediction_result},
50     )

```

Figura 5.30: Realizar una predicción

Este servicio realiza la predicción y devuelve un diccionario, en el que se muestra el tipo de tarea, la salida de la red, e información adicional dependiendo del tipo de tarea.

```

1008 def predict(
1009     node: "Node",
1010     input_data: Any,
1011     project: Optional[dict] = None,
1012 ) -> Any:
1013     """
1014     Local inference with the project "nn.Module".
1015
1016     'input_data' can be a 'dict' (column_name: value), a list of values
1017     in 'input_features' order, or a tensor (rows = batch).
1018     """
1019     project_row = project or _fetch_project_for_node(node.id)
1020     model, model_path = _initiate_model(project_row)
1021     feats = model.get_feats()
1022     metadata = feats.get("metadata") if isinstance(feats.get("metadata"), dict) else {}
1023     metrics = project_row.get("metrics", "categorical_crossentropy")
1024     params = project_row.get("parameters")
1025     if isinstance(params, str):
1026         params = json.loads(params)
1027     task = resolve_task(params, metadata, metrics)
1028
1029     in_features = json.loads(project_row["input_features"]) if isinstance(
1030         project_row["input_features"], str
1031     ) else project_row["input_features"]
1032
1033     net = model.load_model(model_path)
1034     load_trained_weights_into(net, project_row, model_path)
1035     device = get_device()
1036     net = net.to(device)
1037     net.eval()
1038     x = _tensor_from_predict_input(input_data, in_features).to(device)
1039     with torch.no_grad():
1040         out = net(x)
1041     return _format_prediction(out, feats, task)

```

(a) Predicción

```

851 def _format_prediction(
852     row: torch.Tensor,
853     feats: dict,
854     task: str,
855 ) -> dict[str, Any]:
856     if row.dim() >= 2 and row.size(0) > 1:
857         row = row[0:1]
858
859     meta = feats.get("metadata") if isinstance(feats.get("metadata"), dict) else {}
860     out_features = feats.get("output_features") or ["output"]
861     names = out_features
862     if task == "regression":
863         vec = row.squeeze().detach().cpu().numpy()
864         return {
865             "type": "regression",
866             "output": vec.tolist() if vec.ndim > 0 else float(vec),
867             "output_info": {"names": names, "dtype": ["float"] * len(names)},
868         }
869
870     m = (meta.get("labels") if meta else None) or []
871     logits = row.squeeze(0) if row.dim() > 1 else row
872     idx = int(logits.argmax(-1).item()) if logits.dim() > 0 else int(logits.item())
873     probs = F.softmax(logits, dim=-1).detach().cpu().numpy().tolist()
874     label = m[idx] if m and idx < len(m) else idx
875     return {
876         "type": "classification",
877         "output": idx,
878         "labels": list(m) if m else [],
879         "probabilities": probs,
880         "output_info": {
881             "names": names[:1],
882             "dtype": ["str"] if m else ["int"],
883         },
884         "label": label,
885     }

```

(b) Formateo del resultado

Figura 5.31: Código de la predicción.

Después, el caso de uso actualiza la columna *unconfirmed_results*, añadiendo tras la última predicción sin confirmar los parámetros de entrada y de salida de la predicción actual en formato *JSON*.

8. **Confirmar una predicción:** Desde la página de configuración de un proyecto, el usuario puede confirmar predicciones pendientes. Para ello, pulsa el botón “Confirmar Resultados Pendientes”. Esto abre la página en la que aparecen todas las predicciones pendientes de confirmación, con todos los parámetros de entrada y salida. El usuario puede dar por correcto el resultado, o por incorrecto y corregirlo. En cualquier caso, el siguiente procedimiento se realiza con los datos correctos. Una vez el usuario confirma o corrige la predicción, se llama al caso de uso correspondiente con la ronda actual y el resultado. Este añade al final del último *dataset* (calculado igual que en la figura 5.24) el resultado de la predicción.

```

30 def append_confirmed_row(
31     self,
32     pending_item: dict,
33     cur_round: int,
34     in_labels: list[str],
35     out_labels: list[str],
36 ) -> OperationResult[None]:
37     """Append one confirmed pending row as-is to node dataset."""
38     node = pending_item["node"]
39     node_uuid = node.replace("node_", "")
40     path = self.dataset_service.get_last_dataset_path(node_uuid, cur_round, in_labels, out_labels)
41     with open(path, "a", encoding="utf-8") as file_obj:
42         line = ",".join(str(feature) for _, feature in pending_item["data"].items())
43         file_obj.write(line + "\n")
44     return OperationResult(ok=True)
45
46 def append_corrected_row(
47     self,
48     pending_item: dict,
49     corrected_out_features: dict,
50     cur_round: int,
51     in_labels: list[str],
52     out_labels: list[str],
53 ) -> OperationResult[None]:
54     """Append one corrected row preserving input labels plus corrected outputs."""
55     node_uuid = pending_item["node"].replace("node_", "")
56     path = self.dataset_service.get_last_dataset_path(node_uuid, cur_round, in_labels, out_labels)
57     parts = []
58     for key, feature in pending_item["data"].items():
59         if key in in_labels:
60             parts.append(str(feature))
61     for _, value in corrected_out_features.items():
62         parts.append(str(value))
63     with open(path, "a", encoding="utf-8") as file_obj:
64         file_obj.write(",".join(parts) + "\n")
65     return OperationResult(ok=True)

```

Figura 5.32: Confirmar una predicción

9. **Ver métricas del entrenamiento:** Desde la página principal, el usuario puede ver algunas métricas relacionadas con el entrenamiento de sus proyectos. Para ello, selecciona el proyecto deseado y pulsa el botón “Ver métricas del proyecto seleccionado”. Esto llama al caso de uso correspondiente que calcula:

- Tiempo por ronda, extraído directamente del *JSON* del resultado del entrenamiento
- F1 si es clasificación y R^2 si es regresión, utilizando y_true e y_pred de los resultados del entrenamiento.
- Cambios en los *datasets* por ronda, si hay cambios. Esto calcula por cada nodo, los cambios de los distintos *datasets* que le pertenecen y, posteriormente, los suma todos.

```

48 def _per_node_snapshots(files: list[tuple[int, Path]]) -> dict[int, dict]:
49     snapshots: dict[int, dict] = {}
50     sorted_files = sorted(files, key=lambda x: x[0])
51     first_round, first_path = sorted_files[0]
52     with open(first_path, "r", encoding="utf-8") as file_obj:
53         prev_lines = file_obj.readlines()
54     snapshots[first_round] = {
55         "length": max(0, len(prev_lines) - 1),
56         "added": [line.strip().split(",") for line in prev_lines[1:]],
57     }
58     for cur_round, cur_path in sorted_files[1:]:
59         with open(cur_path, "r", encoding="utf-8") as file_obj:
60             cur_lines = file_obj.readlines()
61         prev_counter = Counter(line.strip() for line in prev_lines[1:])
62         cur_counter = Counter(line.strip() for line in cur_lines[1:])
63         added: list[list[str]] = []
64         for row_text, cur_count in cur_counter.items():
65             diff = cur_count - prev_counter.get(row_text, 0)
66             if diff > 0:
67                 added.extend(row_text.split(",") for _ in range(diff))
68         snapshots[cur_round] = {
69             "length": max(0, len(cur_lines) - 1),
70             "added": added,
71         }
72         prev_lines = cur_lines
73     return snapshots

```

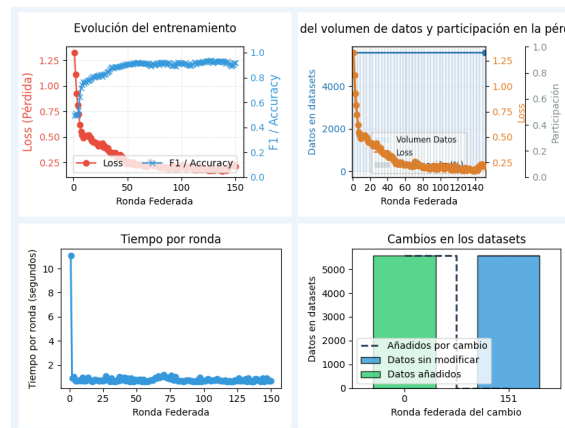
Figura 5.33: Cambios en *dataset* por nodo

Figura 5.34: Pantalla de métricas de un proyecto

10. **Descargar informe con los resultados del entrenamiento:** Desde la página principal, puede descargar un informe en formato PDF con los resultados de los entrenamientos. Para ello, pulsa el botón “Descargar el reporte del proyecto seleccionado”, lo que llama al caso de uso correspondiente con el id del proyecto y la ruta del informe. Esto genera un informe que contiene:

- Información general del proyecto (nombre, id, descripción...).
- Configuración del proyecto (parámetros, estrategia de agregación y función de pérdida).
- Información de cada entrenamiento, con los nodos que han participado y métricas compuestas por ronda (pérdida y R^2 en caso de regresión, pérdida y precisión en clasificación).

- Tiempo total de entrenamiento
- Gráficas y tablas generales del entrenamiento:
 - **Clasificación:** En informes de regresión, se muestra una tabla con precisión, *recall* y F1 por nodo y clase (figura 5.35), una matriz de confusión compuesta de todos los nodos, una tabla con la distribución de clases por nodo y una gráfica en la que se comparan las frecuencias de apariciones reales y predichas de cada clase. Todo esto se calcula con las matrices de confusión compuesta de los nodos en la última ronda.

Nodo	Clase	Precisión	Recall	F1_score
ada69c31-d363-426c-8921-0f198c262138	0	0	0	0
ada69c31-d363-426c-8921-0f198c262138	1	0.9285714285714286	0.3939393939393939	0.5531914893617021
ada69c31-d363-426c-8921-0f198c262138	2	0	0	0
ada69c31-d363-426c-8921-0f198c262138	3	0.8601398601398601	0.9919354838709677	0.9213483146067415
ada69c31-d363-426c-8921-0f198c262138	4	0	0	0
45dab4d0-d5ad-47bf-9d5f-1f36a493f9ec	0	0	0	0
45dab4d0-d5ad-47bf-9d5f-1f36a493f9ec	1	0	0	0
45dab4d0-d5ad-47bf-9d5f-1f36a493f9ec	2	0	0	0
45dab4d0-d5ad-47bf-9d5f-1f36a493f9ec	3	0.96	0.8888888888888888	0.923076923076923
45dab4d0-d5ad-47bf-9d5f-1f36a493f9ec	4	0.9294117647058824	0.9753086419753086	0.9518072289156626
1721f617-880f-4d0d-bdeb-ee8f2b24c473	0	1.0	0.9493670886075949	0.974025974025974
1721f617-880f-4d0d-bdeb-ee8f2b24c473	1	0	0	0
1721f617-880f-4d0d-bdeb-ee8f2b24c473	2	0.0	0	0
1721f617-880f-4d0d-bdeb-ee8f2b24c473	3	0	0	0
1721f617-880f-4d0d-bdeb-ee8f2b24c473	4	0	0	0

Figura 5.35: Tabla de precisión, *recall* y *f1_score* por nodo y por clase

```

476 def get_precision_recall_fscore(client, results_per_round, classes):
477     confusion_matrix = [[0 for _ in range(len(classes)) for _ in range(len(classes))]
478
479     for result in results_per_round:
480         for c in result.get("client_stats") or []:
481             if c.get("client_id") != client or "confusion_matrix" not in c:
482                 continue
483             for i in range(len(classes)):
484                 for j in range(len(classes)):
485                     confusion_matrix[i][j] += c["confusion_matrix"][i][j]
486
487     precision = {}
488     recall = {}
489     f1_score = {}
490
491     for i in range(len(classes)):
492         if sum(confusion_matrix[i]) == 0:
493             precision[classes[i]] = 0
494             recall[classes[i]] = 0
495             f1_score[classes[i]] = 0
496         else:
497             precision[classes[i]] = confusion_matrix[i][i] / sum(confusion_matrix[i])
498             if sum(confusion_matrix[j][i] for j in range(len(classes))) == 0:
499                 recall[classes[i]] = 0
500                 f1_score[classes[i]] = 0
501             else:
502                 recall[classes[i]] = confusion_matrix[i][i] / sum(confusion_matrix[j][i] for j in range(len(classes)))
503             if precision[classes[i]] + recall[classes[i]] == 0:
504                 f1_score[classes[i]] = 0
505             else:
506                 f1_score[classes[i]] = 2 * precision[classes[i]] * recall[classes[i]] / (precision[classes[i]] + recall[classes[i]])
507
508     return precision, recall, f1_score, confusion_matrix

```

Figura 5.36: Cálculo de precisión, *recall* y *f1_score*

- **Regresión:** En informes de regresión, se muestra gráfico de dispersión y distribución de residuos (figura 5.37). Estos se calculan con el resultado predicho y el real compuesto en la última ronda.

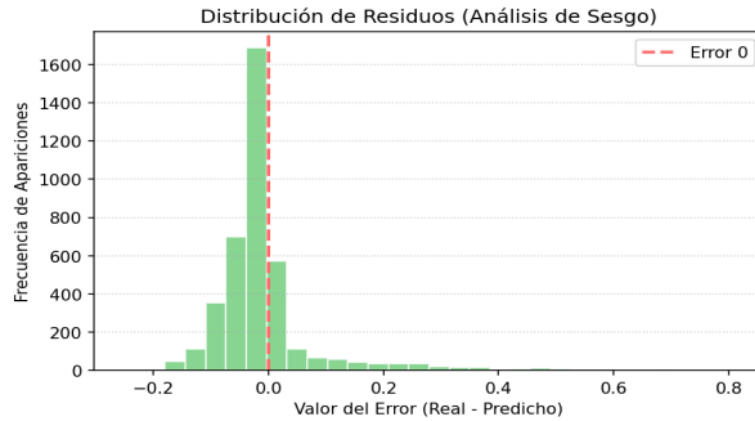


Figura 5.37: Distribución de residuos

```

426 def get_scatter_plot(y_true, y_pred):
427     fig, ax = plt.subplots(figsize=(6, 6))
428
429     ax.scatter(y_true, y_pred, alpha=0.5, color='#4A90E2', edgecolors='white', Label='Predicciones')
430
431     lms = [
432         min(min(y_true), min(y_pred)),
433         max(max(y_true), max(y_pred)),
434     ]
435     ax.plot(lms, lms, color='#FF686B', linestyle='--', linewidth=2, Label='Ideal ($y=x$)')
436
437     ax.set_aspect('equal')
438     ax.set_xlim(lms)
439     ax.set_ylim(lms)
440
441     ax.set_xlabel('Valores Reales')
442     ax.set_ylabel('Predicciones del Modelo')
443     ax.set_title('Gráfico de Dispersión: Real vs. Predicho')
444     ax.legend()
445     ax.grid(True, Linestyle=':', alpha=0.6)
446
447     img_buffer = io.BytesIO()
448     plt.savefig(img_buffer, format='png', bbox_inches='tight', dpi=150)
449     img_buffer.seek(0)
450     plt.close(fig)
451
452     return Image(img_buffer, width=350, height=350)

```

Figura 5.38: Cálculo del gráfico de dispersión

Experimentos

Con el objetivo de comparar el rendimiento de los distintos métodos de agregación implementados en la aplicación, se realizará el siguiente experimento:

- Synthetic:** Para este experimento, se ha generado un *dataset* de manera sintética, utilizando la herramienta presentada en el artículo *LEAF: A Benchmark for Federated Settings* (Caldas et al., 2019). Esta herramienta genera una serie de *datasets* sintéticos, uno para cada cliente especificado, y cada uno tiene relaciones distintas entre sus parámetros de entrada y la salida. Esos datos **Non-IID** son muy comunes entre las distribuciones federadas, por lo que puede ser útil para comprobar el posible rendimiento en un despliegue real. Además, varía el tamaño de cada uno de los *datasets*, lo que es algo bastante habitual en una red de aprendizaje federado real.
- Generación de datos:** Primero, se generan los datos con la herramienta. El comando utilizado para ello es el siguiente:

```
python main.py -num-tasks 500 -num-classes 5 -num-dim 60
```

Esto genera datasets de 500 clientes, con 5 clases en total para predecir y 60 parámetros de entrada.

Además, proporcionan un script para hacer un preprocesado de los datos, permitiendo filtrar según el tamaño mínimo y dividir cada *dataset* seleccionado en entrenamiento y test. El comando utilizado es el siguiente:

```
./preprocess.sh -s niid --sf 1.0 -k 200 -t sample
```

Esto procesa todos los datos de manera que sean *Non-IID*, seleccionando únicamente los que tienen más de 200 para dividir cada uno de ellos en entrenamiento y test.

Este comando genera los *datasets* en formato *.json*, pero para la aplicación, es necesario que estén en formato CSV. Para ello, se ha implementado una herramienta que formatea estos datos al formato deseado:

```
8 all_users_data = defaultdict(lambda: {'x': [], 'y': []})
9
10 for subfolder in ['train', 'test']:
11     full_path = os.path.join(leaf_data_dir, subfolder)
12     if not os.path.exists(full_path): continue
13
14     for filename in os.listdir(full_path):
15         if filename.endswith('.json'):
16             with open(os.path.join(full_path, filename), 'r') as f:
17                 data = json.load(f)
18                 for user in data['users']:
19                     all_users_data[user]['x'].extend(data['user_data'][user]['x'])
20                     all_users_data[user]['y'].extend(data['user_data'][user]['y'])
21
22     if not os.path.exists(output_dir): os.makedirs(output_dir)
23
24     for user, content in all_users_data.items():
25         output_file = os.path.join(output_dir, f"{user}.csv")
26
27         with open(output_file, 'w', newline='') as csvfile:
28             writer = csv.writer(csvfile)
29
30             if content['x']:
31                 num_features = len(content['x'][0])
32                 headers = [f"Feature_{i}" for i in range(num_features)]
33                 if num_features <= 26:
34                     headers = list(string.ascii_uppercase[:num_features])
35
36                 headers.append("Target")
37                 writer.writerow(headers)
38
39             for x, y in zip(content['x'], content['y']):
40                 writer.writerow(x + [y])
```

Figura 6.1: Transformación de *.json* a *.csv* apto para la aplicación

- **Preparación de los proyectos:** Todos los proyectos tendrán los mismos parámetros, salvo el correspondiente a la función de agregación *SCAFFOLD*, ya que esta requiere del uso del optimizador *SGD*. Los parámetros son los siguientes:

Descripción:

Experimento con dataset Synthetic

synthetic_leaf_mlp_model.py

Seleccionar

Tipo de tarea:

classification

Función de pérdida:

categorical_crossentropy

▶ Parámetros

Optimizer:

Adam

Aggregation strategy:

fed_avg

Epochs:

9

Validation split:

0.20

⚠ Batch size:

128

Fraction fit:

0.80

Fraction evaluate:

0.50

Learning rate:

0.0015

Figura 6.2: Parámetros de los experimentos

- Resultados de los experimentos:** Para cada uno de los siguientes experimentos, se ha entrenado 150 rondas.
 - FedAvg*:** Como se puede observar en la gráfica de evolución del entrenamiento (figura 6.3a), en las primeras rondas hay una reducción drástica de la pérdida y gran aumento del *accuracy*, lo que indica que el modelo global captura rápidamente las características de los datos. A partir de la ronda 60 se estabiliza en un *accuracy* de aproximadamente 92 %, en la que se mantiene hasta el final del entrenamiento. En la matriz de confusión de la figura 6.3b, se aprecia que la clase “0” es la que más se confunde, y que se confunde con las clases “2” y “4”. Por su parte, la clase “3” se confunde con la clase “1”, y la “4” con la “2”, aunque esta última en menor medida. Además, mirando la gráfica de la figura 6.3c, se confirma que la clase ‘0’ es infravalorada por el modelo, mientras que las clases “1” y “2”, a pesar de ser las que menos presencia tienen en los *datasets*.

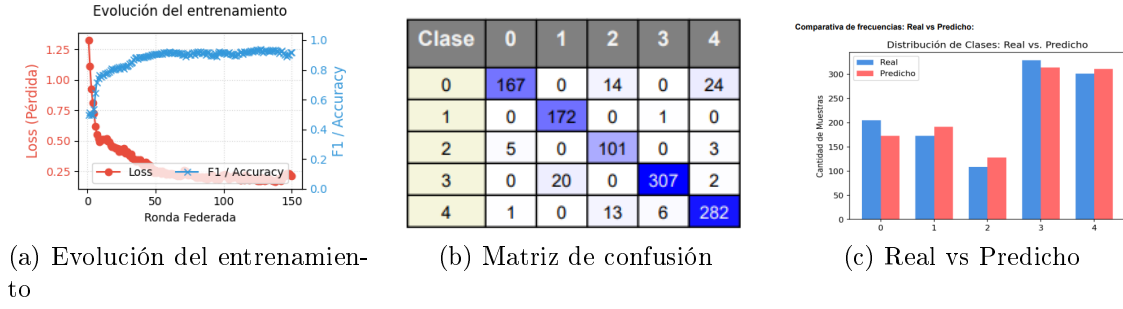


Figura 6.3: Gráficas resultantes del entrenamiento con FedAvg en el dataset Synthetic.

- **FedMed:** En el entrenamiento con *FedMed*, se observa una evolución más suave del entrenamiento, en la que sus puntos forman una línea continua. En este caso, tarda unas 115 rondas en estabilizarse. Todo esto se debe a su naturaleza ya que, al agregar los resultados con la mediana, no toma tan en cuenta los extremos, dando lugar a evoluciones más lentas pero con menos ruido, como se aprecia en la figura 6.4a. En la matriz de confusión de la figura 6.4b, se aprecia que las clases que más se confunden son la clase “0” con la clase “4”, y la clase “2” con las clases “0” y “4”. Cabe destacar que la clase “4” tiene una gran cantidad de falsos positivos. Con la gráfica de la figura 6.4c, podemos corroborar la influencia del algoritmo de agregación, pues las clases que más apariciones tienen en los datos de entrenamiento, son aquellas que más sobrestiman, mientras que los valores más marginales, que menos aparecen en los datos, se infravaloran en las predicciones finales.

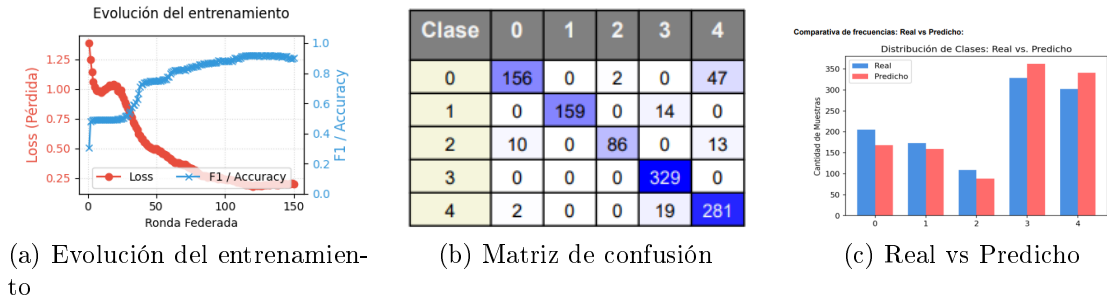


Figura 6.4: Gráficas resultantes del entrenamiento con FedMed en el dataset Synthetic.

- **SCAFFOLD**: En el entrenamiento con *SCAFFOLD*, se observa una evolución más inestable, con picos muy bruscos en las rondas 50 y 100, como se puede apreciar en la figura 6.5a. Esto se debe a que el algoritmo trata de compensar el sesgo de los datos locales, y cuando hay clientes que tienen clases completamente opuestas, las variables de control utilizadas para corregir el *client drift* pueden provocar correcciones agresivas. En la figura 6.1 se puede ver que hay clientes con los datos muy sesgados, la gran mayoría o la totalidad de sus datos pertenecen a una sola clase. Después de este último pico en la ronda 100, tanto la pérdida como la precisión comienzan a estabilizarse, convergiendo en valores superiores de pérdida y precisión inferior a la del resto de algoritmos. En la matriz de confusión de la figura 6.5b, se puede observar que las clases que más se confunden son la clase “3” con la clase “1” y las clases “4” y “2” con la clase “0”. La clase con más falsos positivos es la clase “0”, lo que se puede apreciar también en la gráfica de la figura 6.5c.

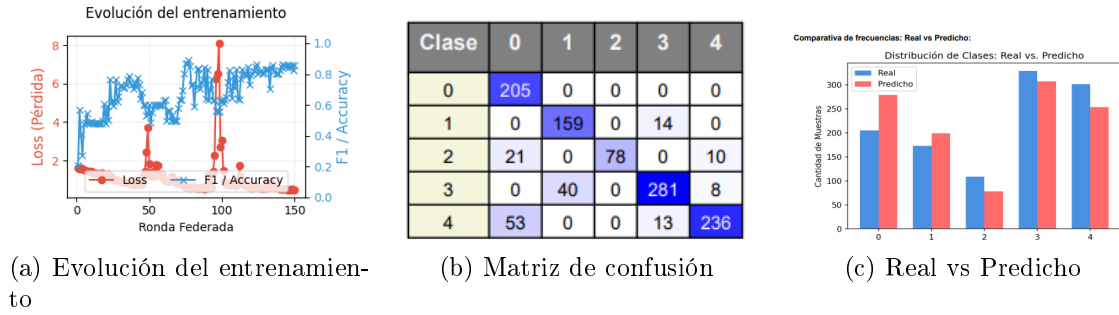


Figura 6.5: Gráficas resultantes del entrenamiento con SCAFFOLD en el dataset Synthetic.

Nodo (ID)	Clase 0	Clase 1	Clase 2	Clase 3	Clase 4
e0541366...	0	14	0	143	0
c5419f25...	0	0	0	50	85
9f53aa7a...	75	0	4	0	0
59c7f30e...	0	0	78	0	0
fa06a3c5...	0	0	0	0	129
81d851ce...	0	0	10	0	35
8e0f7648...	130	0	17	0	53
d8663387...	0	0	0	96	0
9fc3ab94...	0	158	0	0	0
ac86de70...	0	1	0	40	0

Tabla 6.1: Distribución de clases Non-IID por cada nodo del experimento Synthetic.

- **SSFed:** En el entrenamiento con *SSFed*, se observa una gran evolución en las primeras rondas, estabilizándose cerca de la ronda 50. A partir de esta ronda, la precisión aumenta lentamente y, por su parte, la pérdida se reduce también lentamente, con ciertas oscilaciones. Ambas alcanzan sus mejores valores hacia el final del entrenamiento, como se puede ver en la figura 6.6a. En la matriz de confusión (figura 6.6b), se observa una gran precisión, con confusiones mínimas. La clase “0” se confunde con la clase “2” y la clase “3” se confunde con las clases “1” y “4”. La estabilidad y precisión del entrenamiento se puede observar en la figura 6.6c, en la que no destaca especialmente la sobrestimación de ninguna de las clases. Sí se puede apreciar que la clase “3” es infravalorada en mayor medida que el resto a la hora de predecir, sin embargo, la diferencia con el resto es prácticamente inapreciable.

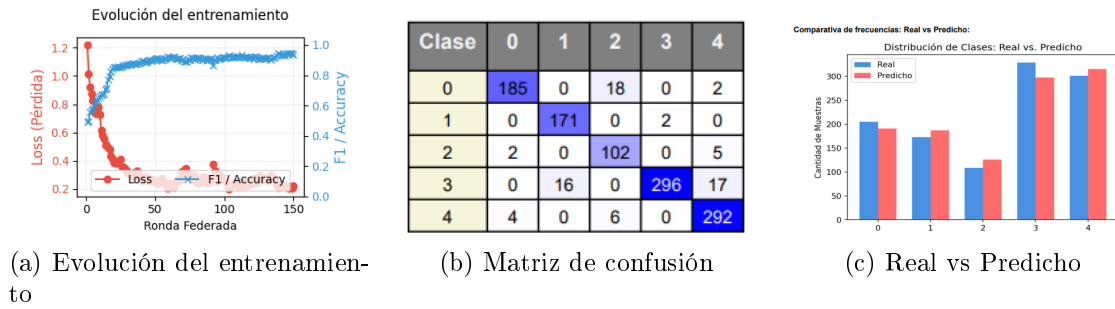


Figura 6.6: Gráficas resultantes del entrenamiento con SSFed en el dataset Synthetic.

- **Conclusión:** A continuación, se muestra una gráfica con la comparación entre la precisión de cada uno de los métodos de agregación en la última ronda:

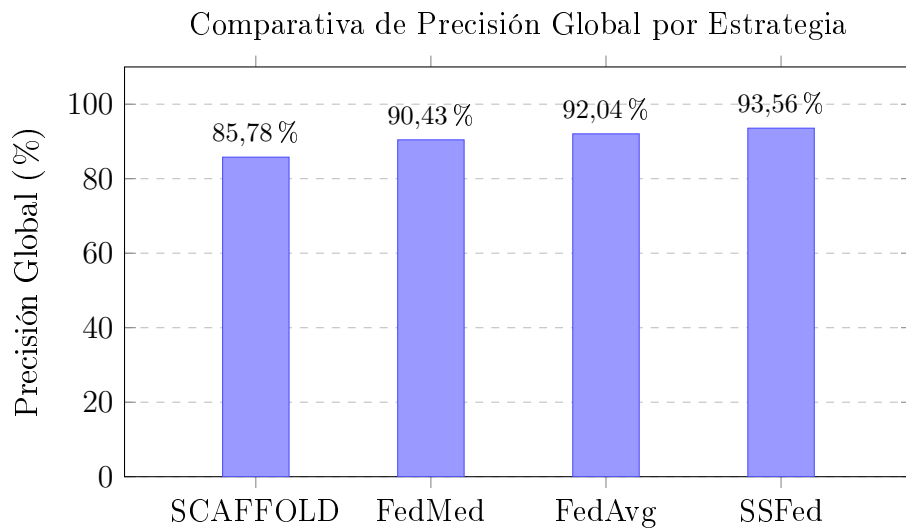


Figura 6.7: Comparativa de la precisión global en porcentaje para cada algoritmo.

Una vez vistos y analizados todos los resultados, se puede llegar a la conclusión de que, aunque las diferencias son pequeñas, dependiendo del contexto, la utilización de una estrategia u otra debe depender de la heterogeneidad y ruido del entorno. *FedAvg* tiende a ser más sensible al ruido, pero en situaciones en las que los datos están distribuidos de forma heterogénea pero no extrema, es decir, los clientes aún comparten suficientes características comunes, es muy efectivo. Además, es un algoritmo muy eficiente. Su alta sensibilidad al ruido limita su efectividad en casos extremos. Por otro lado, *FedMed* es menos sensible al ruido, por lo que funciona mejor en contextos más sesgados. *SCAFFOLD* es un algoritmo más complejo que ajusta las desviaciones de los clientes, teniendo mejores resultados en entornos Non-IID más extremos, a costa de una mayor inestabilidad inicial e ineficiencia. *SSFed* ha demostrado ser el más efectivo en este experimento; al basar su agregación en la significancia estadística de cada nodo, prioriza las actualizaciones que convergen hacia un modelo global común y mitigan el impacto de las desviaciones locales. Esto lo posiciona como la solución más equilibrada y resiliente ante la alta heterogeneidad de los datos.

Conclusiones y Trabajo Futuro

7.1. Conclusiones

El aprendizaje federado es un paradigma novedoso, cuyas características en lo relativo a la privacidad y el manejo de grandes cantidades de datos serán muy importantes y utilizadas del presente en adelante.

En este contexto, FedBench se presenta como una herramienta innovadora que integra la preparación de simulaciones de entornos de aprendizaje federado, el entrenamiento distribuido y el posterior análisis de los resultados de estos entrenamientos. Para hacer esto posible, se han desarrollado ciertas funcionalidades que permiten al usuario crear proyectos, añadir sus propios *datasets* y modelos a un proyecto, añadir nodos a un proyecto, entrenar los nodos y generar informe en base a estos entrenamientos. Además, se proporciona al usuario una interfaz intuitiva que permite a perfiles sin un gran conocimiento técnico de este paradigma puedan realizar estudios sobre su rendimiento de manera sencilla y eficiente.

El código fuente de FedBench se puede encontrar en el siguiente repositorio de GitHub: **Código fuente**

7.2. Trabajo Futuro

A pesar de que la aplicación cumple con los objetivos planteados inicialmente, la limitación de tiempo propia de la realización del trabajo obliga a priorizar la implementación y estabilidad del núcleo funcional inicial. Debido a la naturaleza innovadora del proyecto y a la limitación mencionada, han surgido nuevas líneas de evolución y desarrollo que no han podido ser implementadas en esta primera versión. A continuación, se mencionan estas funcionalidades, que serán desarrolladas en el futuro:

7.2.1. Funcionalidades futuras

- **Migración a una arquitectura distribuida:** Con la experiencia adquirida durante la realización de este trabajo, considero que el verdadero potencial

de la aplicación está en ofrecer un entorno independiente de su dispositivo al usuario, con una gran capacidad de procesamiento para simular aprendizajes con más clientes. Esto implica separar definitivamente la lógica de la aplicación, dejando solo la interfaz que realiza llamadas a la *API* del servidor. Con ello, se consigue además que la aplicación sea más ligera y requiera mucha menor capacidad de computación por parte del usuario. Esto tendría otras implicaciones de seguridad, principalmente en el punto de la carga y ejecución dinámica de código del usuario. Esto se trata en el punto siguiente.

- **Creación interactiva de modelos y mejora de carga dinámica:** Con el objetivo de proporcionar al usuario un método para añadir un modelo sin necesidad de codificarlo manualmente, se propone la creación de una herramienta *No-Code* para definir modelos de manera interactiva. De esta forma, además se evita la posible ejecución de código malicioso en los modelos proporcionados por el usuario. Con intención de resolver también estos problemas, se investigarán formas de ejecutar los modelos en entornos aislados (*sandboxing*) para mitigar los riesgos de ejecución maliciosa en el servidor, además del análisis estático del código previo a esta ejecución.
- **Soporte para modelos *TensorFlow*:** Hoy en día, la mayoría de redes desplegadas en producción son diseñadas con *TensorFlow*. Por ello, con el objetivo de proporcionar al usuario una simulación más realista y aplicable para su despliegue, se añadirá la compatibilidad con modelos de este *framework*, además de los de *Pytorch*. Para ello, se investigará el uso de formatos estándar como *ONNX* (*Open Neural Network Exchange*), facilitando el uso de modelos independientemente del *framework* utilizado originalmente.
- **Personalización de informes:** Con el objetivo de dar al usuario la información que necesite en los informes, se implementará un módulo avanzado de reportes que permita la personalización de las métricas mostradas. Esta funcionalidad dará al usuario la opción de elegir qué métricas mostrar al generar el informe y de cambiar los formatos en los que se muestran.
- **Ampliación de estrategias disponibles:** El aprendizaje federado es un paradigma muy reciente y, por lo tanto, está en constante innovación. Por ello, con el objetivo de mantener la aplicación actualizada con el estado del paradigma, se añadirán tanto estrategias ya consolidadas, como distintas estrategias emergentes que puedan ir apareciendo.
- **Asistente analítico basado en IA:** Esta funcionalidad permitirá al usuario analizar de forma sencilla y rápida los resultados de sus proyectos sin necesidad de comparar todos los informes manualmente, utilizando consultas en lenguaje natural. El usuario podrá crear un *chat* con un agente de IA, que utilizará una arquitectura de Generación Aumentada por Recuperación (*RAG*). Utilizando el protocolo *MCP* (*Model Context Protocol*), podrá consultar los informes del usuario permitiendo identificar patrones, comparar métricas y obtener conclusiones sin intervención manual.

- **Tratamiento de imágenes:** Uno de los usos más comunes del aprendizaje federado es la clasificación de imágenes. En esta aplicación, por falta de tiempo no se implementó la compatibilidad con imágenes y no es posible utilizarlas directamente, siendo necesario un tratamiento previo complejo. Para facilitar al usuario esta tarea, se implementará un módulo para el tratamiento de *datasets* de imágenes que estandarice las imágenes y proporcione la posibilidad de utilizar técnicas como el aumento de datos para mejorar el entrenamiento local, facilitando al usuario el despliegue de modelos de visión artificial.
- **Simulación de entornos no ideales:** En un entorno federado real, los entrenamientos no son ideales: hay clientes cuyos *datasets* contienen ruido, se desconectan inesperadamente durante el entrenamiento, tienen conexiones más lentas unos que otros... Por este motivo, puede ser interesante ofrecer la integración de parámetros para controlar estos factores, con el objetivo de simulación más realista y representativa de una situación real.

Conclusions and Future Work

7.3. Conclusions

Federated Learning is an innovative paradigm, and its features relating to privacy and the handling of large amounts of data will be of a great importance and widely used from now on.

In this context, FedBench is presented as an innovative tool that integrates the preparation of federated learning simulations, distributed training and subsequent result analysis. To achieve this, several features were implemented, allowing users to create projects, incorporate their own datasets and models to a project, add nodes to a project and generate a report based on these trainings. Furthermore, the intuitive interface allows users with limited technical knowledge in this paradigm to conduct performance studies easily and efficiently.

7.4. Future work

While the application fulfills the initial objectives, the project's time constraints led to a prioritization of the implementation and stability of the initial functional core. Given the innovative nature of the project and aforementioned limitation, new research directions and development pathways have been identified for future iterations of the software. The following sections outline these future functionalities:

7.4.1. Future features

- **Migration to a distributed architecture:** With the experience acquired during this project's development, the application's full potential lies on providing an environment independent of the users' hardware, thereby enabling training simulations with more clients. This implies decoupling the application logic from the client side, leaving only a frontend interface that interacts with the server through API calls. This requires significantly less computing power from the user. This transition introduces significant security implications, primarily related to the dynamic loading and execution of external code. This is addressed in the next item.

- **Interactive model creation and dynamic loading optimisation:** To enable model definition without manual coding, we propose the development of a *No-Code* tool for interactive model design. This approach minimizes the risks associated with executing malicious user-provided code. address remaining security concerns, future work will focus on isolated execution environments (*sandboxing*) and pre-execution static code analysis to mitigate potential server-side threats.
- **Support for *TensorFlow* models:** Currently, most networks deployed in production are designed using *TensorFlow*. Therefore, with the aim of providing users with a more realistic simulation that is applicable to their deployment, compatibility with models developed using this framework will be added, in addition to those from *PyTorch*. To this end, the use of standard formats such as *ONNX* (*Open Neural Network Exchange*) will be investigated, facilitating the use of models regardless of the framework originally used.
- **Report customisation:** To ensure users receive the information they need in their reports, an advanced reporting module will be implemented that allows users to customise the metrics displayed. This feature will enable users to choose which metrics to display when generating a report and to change the formats in which they are presented.
- **Expansion of available strategies:** Federated learning is a very recent paradigm and is therefore constantly evolving. Consequently, in order to keep the application up to date with the latest developments in the field, we will be adding both established strategies and various emerging strategies as they become available.
- **AI-powered analytical assistant:** This feature will enable users to analyse their project results quickly and easily, without the need to manually compare all reports, using natural language queries. Users will be able to create a chat with an AI agent, which will use a Retrieval-Augmented Generation (RAG) architecture. Using the Model Context Protocol (MCP), the agent will be able to query the user's reports, enabling it to identify patterns, compare metrics and draw conclusions without human intervention.
- **Image processing:** One of the most common uses of federated learning is image classification. In this application, due to time constraints, image support was not implemented and it is not possible to use images directly; instead, complex pre-processing is required. To simplify this task for the user, a module will be implemented for processing image *datasets* that standardises the images and provides the option to use techniques such as data augmentation to improve local training, thereby making it easier for the user to deploy computer vision models.
- **Simulation of non-ideal environments:** In a real federated environment, training sessions are not ideal: some clients have *datasets* which contain noise,

some unexpectedly disconnect during training, and some have slower connections than others... For this reason, it may be worthwhile to integrate parameters to control these factors, with the aim of achieving a more realistic simulation that closely represents a real-world situation.

Bibliografía

*¡Libros! ¡Libros! Hace aquí una palabra
mágica que equivale a decir: “amor, amor”, y
que debían los pueblos pedir como piden pan*

Federico García Lorca

- ALSENANI, Y. Ssfed: Statistical significance aggregation algorithm in federated learning. *International Journal of Advanced Computer Science and Applications*, vol. 16, 2025.
- CALDAS, S., DUDDU, S. M. K., WU, P., LI, T., KONEČNÝ, J., MCMAHAN, H. B., SMITH, V. y TALWALKAR, A. Leaf: A benchmark for federated settings. 2019.
- FLOWER LABS. Flower framework documentation. <https://flower.ai/docs/framework/>, 2026. Documentación oficial de Flower para Aprendizaje Federado.
- HARD, A., RAO, K., MATHEWS, R., RAMASWAMY, S., BEAUFAYS, F., AUGENSTEIN, S., EICHNER, H., KIDDON, C. y RAMAGE, D. Federated learning for mobile keyboard prediction. 2019.
- INTERNATIONAL TELECOMMUNICATION UNION. Facts and figures 2023: Mobile phone ownership. Statistics report, ITU Publications, Geneva, Switzerland, 2023.
- KARIMIREDDY, S. P., KALE, S., MOHRI, M., REDDI, S. J., STICH, S. U. y SURESH, A. T. Scaffold: Stochastic controlled averaging for federated learning. 2021.
- LI, T., SAHU, A. K., ZAHEER, M., SANJABI, M., TALWALKAR, A. y SMITH, V. Federated optimization in heterogeneous networks. 2020.
- MCMAHAN, H. B., MOORE, E., RAMAGE, D., HAMPSON, S. y Y ARCAS, B. A. Communication-efficient learning of deep networks from decentralized data. 2023.
- NILSSON, A., SMITH, S., ULM, G., GUSTAVSSON, E. y JIRSTRAND, M. A performance evaluation of federated learning algorithms. páginas 1–8. 2018.
- PYTHON SOFTWARE FOUNDATION. tkinter — python interface to tcl/tk. <https://docs.python.org/es/3.12/library/tkinter.html>, 2023. Documentación oficial Python sobre Tkinter.

- PYTORCH FOUNDATION. PyTorch documentation. <https://docs.pytorch.org/docs/2.10/index.html>, 2025. Documentación oficial de PyTorch para la versión 2.10.0.
- REDDI, S., CHARLES, Z., ZAHEER, M., GARRETT, Z., RUSH, K., KONEČNÝ, J., KUMAR, S. y MCMAHAN, H. B. Adaptive federated optimization. 2021.
- SCIKIT-LEARN DEVELOPERS. scikit-learn API reference (version 1.8). <https://scikit-learn.org/stable/api/index.html>, 2026. Documentación oficial de scikit-learn.
- SHELLER, M., EDWARDS, B., REINA, G., MARTIN, J., PATI, S., KOTROTSOU, A., MILCHENKO, M., XU, W., MARCUS, D., COLEN, R. y BAKAS, S. Federated learning in medicine: facilitating multi-institutional collaborations without sharing patient data. *Scientific Reports*, vol. 10, 2020.
- SQLITE CONSORTIUM. About SQLite. <https://www.sqlite.org/about.html>, 2025. Descripción general de las características del software.
- WANG, J., LIU, Q., LIANG, H., JOSHI, G. y POOR, H. V. Tackling the objective inconsistency problem in heterogeneous federated optimization. 2020.
- YIN, D., CHEN, Y., RAMCHANDRAN, K. y BARTLETT, P. Byzantine-robust distributed learning: Towards optimal statistical rates. 2021.

Casos de uso

A.1. Actores

- **Usuarios no registrados:** Usuarios que no han iniciado sesión. Solamente tienen acceso a la página de *Login*.
- **Usuarios registrados:** Usuarios que han iniciado sesión. Tienen acceso a la página principal, su gestión de cuentas y la gestión de proyectos.
- **Usuario administrador:** Usuario administrador del sistema. Tienen acceso a la página principal, su gestión de cuentas; pueden dar de alta y baja a nodos y tienen acceso a la gestión de usuarios.

A.2. Casos de uso

Tabla A.1: Caso de Uso de Login

Requisito	Login
Identificador	1.1
Prioridad	Alta
Descripción	Los usuarios con una cuenta creada pueden iniciar sesión al iniciar la aplicación.
Precondición	No estar registrado y tener una cuenta creada.
Entrada	Nombre de usuario, contraseña
Salida	NA
Secuencia normal	1. El usuario accede a la aplicación, donde se encuentra la pantalla de inicio de sesión. 2. La aplicación solicita el nombre del usuario y su contraseña. 3. El usuario introduce los datos y hace <i>click</i> en “Iniciar sesión” 4. La aplicación comprueba la corrección de los datos. 5. La aplicación pasa a la pantalla principal.
Postcondición	Se ha iniciado sesión.
Excepciones	■ En caso de no existir el nombre de usuario, se mostrará un mensaje de error. ■ En caso de no ser la contraseña correcta para el usuario, se mostrará un mensaje de error.
Comentarios	NA
Actores	Usuarios no registrados.

Tabla A.2: Caso de Uso de Logout

Requisito	Logout
Identificador	1.2
Prioridad	Alta
Descripción	Los usuarios registrados pueden cerrar sesión.
Precondición	Ser usuario registrado.
Entrada	NA
Salida	NA
Secuencia normal	1. Desde el menú principal, el usuario hace <i>click</i> en “Mi Cuenta” y se muestra la página de perfil. 2. El usuario hace <i>click</i> en “Cerrar sesión”. 3. La aplicación despliega un diálogo de confirmación. 4. El usuario confirma el cierre de sesión. 5. La aplicación cierra la sesión del usuario y vuelve a la pantalla de inicio de sesión.
Postcondición	Se ha cerrado la sesión y el usuario pasa a ser no registrado.
Excepciones	NA
Comentarios	NA
Actores	Usuarios registrados.

Tabla A.3: Caso de Uso de Modificar perfil

Requisito	Modificar perfil
Identificador	1.3
Prioridad	Baja
Descripción	Los usuarios registrados pueden modificar algunos datos de su perfil.
Precondición	Ser usuario registrado.
Entrada	Nombre de usuario, contraseña, confirmación de contraseña, frase de recuperación y confirmación de frase de recuperación.
Salida	NA
Secuencia normal	1. Desde el menú principal, el usuario hace <i>click</i> en “Mi Cuenta” y se muestra la página de perfil. 2. El usuario hace <i>click</i> en “Modificar cuenta”. 3. La aplicación muestra la pantalla con el formulario para modificar el nombre de usuario, la contraseña y la frase de recuperación de la cuenta. 4. El usuario modifica los campos deseados y hace <i>click</i> en “Confirmar cambios”. 5. La aplicación muestra un diálogo con el siguiente mensaje: “Los datos se han modificado correctamente” y vuelve a la pantalla de perfil.
Postcondición	Se han modificado los datos del usuario.
Excepciones	NA
Comentarios	Si el campo de contraseña o el de frase de recuperación están vacíos, no se modifican.
Actores	Usuarios registrados.

Tabla A.4: Caso de Uso de Baja de usuario

Requisito	Baja de usuario
Identificador	1.4
Prioridad	Media
Descripción	El usuario administrador puede dar de baja a un usuario.
Precondición	Ser usuario administrador y usuario a eliminar existente.
Entrada	Usuario a eliminar
Salida	NA
Secuencia normal	1. Desde el menú principal, el usuario hace <i>click</i> en “Gestión de usuarios” y se despliega el listado de usuarios. 2. El usuario selecciona el usuario deseado y se despliega la información del usuario a eliminar. 3. El usuario hace <i>click</i> en “Eliminar Usuario”. 4. La aplicación despliega un diálogo de confirmación. 5. El usuario confirma y el usuario seleccionado es dado de baja.
Postcondición	Usuario dado de baja, proyectos asociados a él eliminados y los nodos de sus proyectos liberados.
Excepciones	NA
Comentarios	NA
Actores	Usuario administrador.

Tabla A.5: Caso de Uso de Registro de usuario

Requisito	Registro de usuario
Identificador	1.5
Prioridad	Alta
Descripción	Los usuarios nuevos pueden registrarse en la aplicación.
Precondición	Ser usuario no registrado.
Entrada	Usuario, contraseña y frase de recuperación.
Salida	NA
Secuencia normal	1. En la pantalla de inicio de sesión, el usuario hace <i>click</i> en “No tengo cuenta”. 2. Se muestra el formulario de registro con los campos “Usuario”, “Contraseña”, “Confirmar contraseña”, “Frase de recuperación”, “Confirmar frase”. 3. La aplicación valida los datos y pasa a la pantalla de inicio de sesión.
Postcondición	Se ha registrado al usuario.
Excepciones	■ En caso de que el usuario ya exista, se mostrará un mensaje de error. ■ En caso de que las contraseñas no coincidan, se mostrará un mensaje de error. ■ En caso de que las frases de recuperación no coincidan, se mostrará un mensaje de error.
Comentarios	NA
Actores	Usuarios no registrados.

Tabla A.6: Caso de Uso de Recuperar contraseña

Requisito	Recuperar contraseña
Identificador	1.6
Prioridad	Media
Descripción	Los usuarios pueden cambiar su contraseña.
Precondición	Ser usuario no registrado con una cuenta registrada en la aplicación.
Entrada	Usuario, frase de recuperación y nueva contraseña.
Salida	NA
Secuencia normal	1. En la pantalla de inicio de sesión, el usuario hace <i>click</i> en “Olvidé mi contraseña”. 2. Se muestra el formulario de recuperación con el campo “Usuario”. 3. La aplicación comprueba la validez y muestra el formulario con los campos “Frase de recuperación”, “Nueva contraseña” y “Confirmar nueva contraseña”. 4. La aplicación valida los datos, muestra el mensaje “Contraseña actualizada correctamente” y pasa a la pantalla de inicio de sesión.
Postcondición	Se ha cambiado la contraseña del usuario.
Excepciones	■ En caso de que el usuario no exista, se mostrará un mensaje de error. ■ En caso de que la frase de recuperación no coincida con la registrada para el usuario, se mostrará un mensaje de error. ■ En caso de que las contraseñas no coincidan, se mostrará un mensaje de error.
Comentarios	NA
Actores	Usuarios no registrados.

Tabla A.7: Caso de Uso de Alta de nodo

Requisito	Alta de nodo
Identificador	2.1
Prioridad	Alta
Descripción	El usuario administrador puede registrar nuevos nodos en el sistema
Precondición	Ser usuario administrador.
Entrada	NA
Salida	NA
Secuencia normal	1. El usuario accede a la aplicación, donde se encuentra la lista de nodos. 2. El usuario hace <i>click</i> en “Agregar nodo”. 3. La aplicación muestra el mensaje: “Nodo registrado correctamente” y añade el nuevo nodo a la lista.
Postcondición	Nuevo nodo registrado en la aplicación.
Excepciones	NA
Comentarios	Se genera un UUID aleatorio al crearlo para identificarlo.
Actores	Usuario administrador

Tabla A.8: Caso de Uso de Baja de nodo

Requisito	Baja de nodo
Identificador	2.2
Prioridad	Alta
Descripción	El usuario administrador puede eliminar nodos existentes de la aplicación.
Precondición	El usuario es administrador y el nodo a eliminar existe.
Entrada	Nodo a eliminar.
Salida	NA
Secuencia normal	1. El usuario accede a la aplicación, donde se encuentra el menú principal. 2. El usuario hace <i>click</i> en el nodo a eliminar. 3. El usuario hace <i>click</i> en “Eliminar nodo seleccionado”. 4. Si el nodo pertenece a un proyecto, la aplicación solicita confirmación de la eliminación. 5. El usuario confirma la eliminación y la aplicación elimina el nodo de la base de datos y de la lista.
Postcondición	Nodo eliminado de la aplicación.
Excepciones	NA
Comentarios	NA
Actores	Usuario administrador

Tabla A.9: Caso de Uso de Creación de un proyecto

Requisito	Creación de un proyecto
Identificador	3.1
Prioridad	Alta
Descripción	El usuario puede crear un proyecto nuevo con una red de nodos.
Precondición	El usuario está registrado.
Entrada	Nombre, descripción, modelo a utilizar, tipo de tarea, función de pérdida, nodos, parámetros del algoritmo, estrategia de agregación.
Salida	NA
Secuencia normal	1. Desde la página principal, el usuario hace <i>click</i> en “Crear proyecto”. 2. La aplicación muestra el formulario a rellenar para la creación del proyecto. 3. El usuario introduce el nombre y la descripción del proyecto, selecciona el modelo a utilizar, el tipo de tarea, la función de pérdida a utilizar, los nodos disponibles que pertenecerán al proyecto, los parámetros del algoritmo, y la estrategia de agregación a utilizar; y hace <i>click</i> en “Crear”. 4. La aplicación confirma la validez de los datos y crea el proyecto.
Postcondición	Nuevo proyecto creado con las configuraciones seleccionadas.
Excepciones	■ En caso de introducir un dato en el formato no válido, se mostrará un mensaje de error. ■ En caso de dejar alguno de los datos en blanco, salvo la descripción y los nodos, se mostrará un mensaje de error y se volverá al formulario.
Comentarios	NA
Actores	Usuario registrado

Tabla A.10: Caso de Uso de Configuración de un proyecto

Requisito	Configuración de un proyecto
Identificador	3.2
Prioridad	Media
Descripción	El usuario puede modificar algunos aspectos de un proyecto, como el nombre, la descripción o los nodos implicados en el proyecto.
Precondición	El usuario está registrado y el proyecto a modificar existe.
Entrada	Datos a modificar.
Salida	NA
Secuencia normal	1. Desde la página principal, el usuario selecciona un proyecto y hace <i>click</i> en “Configurar el proyecto seleccionado”. 2. La aplicación muestra un formulario en el que se da la opción de modificar el nombre, la descripción o los nodos implicados en el proyecto. 3. El usuario modifica los datos que desee y hace <i>click</i> en “Confirmar”. 4. La aplicación verifica la corrección de los datos y modifica el proyecto.
Postcondición	Proyecto modificado con la nueva configuración.
Excepciones	■ En caso de dejar alguno de los datos en blanco, salvo la descripción y los nodos, se mostrará un mensaje de error y se volverá al formulario.
Comentarios	NA
Actores	Usuario registrado

Tabla A.11: Caso de Uso de Realización de predicción

Requisito	Realización de predicción
Identificador	3.3
Prioridad	Alta
Descripción	El usuario puede realizar una predicción de un resultado en base a unos datos de entrada.
Precondición	El usuario está registrado y el proyecto existe.
Entrada	Datos de entrada con los que realizar la predicción.
Salida	Resultado de la predicción.
Secuencia normal	1. Desde la página principal, el usuario selecciona el proyecto deseado y hace <i>click</i> en “Realizar predicción”. 2. La aplicación muestra un diálogo para que el usuario introduzca los datos de entrada y el nodo en el que se quiere realizar la predicción. 3. El usuario introduce los datos y hace <i>click</i> en “Confirmar”. 4. La aplicación realiza las predicciones y guarda los resultados en “Confirmaciones pendientes” en el proyecto.
Postcondición	Predicciones realizadas y añadidas en “Confirmaciones pendientes” del proyecto seleccionado.
Excepciones	■ En caso de haber algún error al realizar la predicción, se mostrará un mensaje de error. ■ En caso de dejar algún campo de entrada vacío, se mostrará un mensaje de error. ■ En caso de no coincidir con el número de parámetros de entrada del proyecto, se mostrará un mensaje de error.
Comentarios	NA
Actores	Usuario registrado

Tabla A.12: Caso de Uso de Añadir *dataset*

Requisito	Añadir <i>dataset</i>
Identificador	3.4
Prioridad	Alta
Descripción	El usuario puede añadir un <i>dataset</i> a un nodo de un proyecto para el próximo entrenamiento.
Precondición	El usuario está registrado, el proyecto existe y tiene algún nodo asociado.
Entrada	Dataset a añadir.
Salida	NA
Secuencia normal	1. Desde la página principal, el usuario selecciona la configuración del proyecto deseado y hace <i>click</i> en “Añadir dataset”. 2. La aplicación muestra un diálogo para añadir el <i>dataset</i> al proyecto. 3. El usuario selecciona el nodo en el que añadirlo, introduce el fichero del <i>dataset</i> y hace <i>click</i> en “Confirmar”. 4. La aplicación guarda el <i>dataset</i> y muestra el mensaje “El dataset fue añadido con éxito”.
Secuencias alternativas	■ A1: El nodo ya tiene un <i>dataset</i>: 1. El sistema detecta que el <i>dataset</i> ya existe y la última ronda coincide. 2. El sistema añade la información del <i>dataset</i> nuevo al final del dataset existente. 3. Se vuelve al paso 4 de la secuencia principal. ■ A2: El nodo tiene un <i>dataset</i> pero no coincide con la última ronda de entrenamiento: 1. El sistema detecta que el <i>dataset</i> ya existe y la última ronda no coincide con el número del dataset. 2. El sistema duplica el dataset antiguo y cambia el número al de la nueva ronda. 3. El sistema añade la información del <i>dataset</i> nuevo al final del dataset duplicado. 4. Se vuelve al paso 4 de la secuencia principal.
Postcondición	<i>Dataset</i> añadido al nodo.

Excepciones	▪ En caso de haber algún error al realizar el <i>parse</i> de los ficheros, se mostrará un mensaje de error. ▪ En caso de no tener permisos para leer alguno de los ficheros, se mostrará un mensaje de error. ▪ En caso de no existir alguno de los ficheros, se mostrará un mensaje de error.
Comentarios	NA
Actores	Usuario registrado

Tabla A.13: Caso de Uso de Confirmación de resultados de un proyecto

Requisito	Confirmación de resultados de un proyecto
Identificador	3.5
Prioridad	Alta
Descripción	El usuario puede confirmar los resultados de la predicción para el posterior entrenamiento.
Precondición	El usuario está registrado y se han realizado predicciones sin confirmar resultados.
Entrada	Confirmación de la predicción.
Salida	NA
Secuencia normal	1. Desde la página principal, el usuario selecciona las opciones del proyecto deseado y hace <i>click</i> en “Confirmaciones pendientes”. 2. La aplicación muestra la lista de predicciones pendientes de confirmación. 3. El usuario hace <i>click</i> en el botón “Correcta” para confirmar el resultado correcto. 4. La aplicación guarda los resultados en la base de datos.
Secuencias alternativas	1. El usuario hace <i>click</i> en el botón “Incorrecta”. 2. El sistema muestra un diálogo para que el usuario introduzca los resultados correctos. 3. El usuario introduce los resultados correctos de la predicción y hace <i>click</i> en “Guardar”. 4. La aplicación guarda el resultado corregido en la base de datos.
Postcondición	Datos añadidos al <i>dataset</i> de entrenamiento del nodo que realizó la predicción.
Excepciones	NA
Comentarios	Los datos se insertan en el <i>dataset</i> del nodo de la misma manera que en el 3.4,
Actores	Usuario registrado

Tabla A.14: Caso de Uso de Descarga de informe del proyecto

Requisito	Descarga de informe del proyecto
Identificador	3.6
Prioridad	Media
Descripción	El usuario puede descargar un informe de los resultados de los entrenamientos de un proyecto en formato PDF.
Precondición	El usuario está registrado, el proyecto existe y se ha realizado algún entrenamiento.
Entrada	Ruta destino.
Salida	Fichero PDF con el informe.
Secuencia normal	1. Desde la página principal, el usuario selecciona el proyecto deseado y hace <i>click</i> en “Descargar informe PDF”. 2. La aplicación muestra un diálogo para que el usuario seleccione la ruta del fichero. 3. El usuario selecciona la ruta y hace <i>click</i> en “Confirmar”. 4. La aplicación guarda el informe en la ruta seleccionada.
Postcondición	Informe realizado y guardado en la ruta seleccionada.
Excepciones	■ En caso de no tener permisos de escritura en el directorio, se mostrará un mensaje de error. ■ En caso de no existir el directorio destino, se mostrará un mensaje de error.
Comentarios	NA
Actores	Usuario registrado

Tabla A.15: Caso de Uso de Eliminación de un proyecto

Requisito	Eliminación de un proyecto.
Identificador	3.7
Prioridad	Alta
Descripción	El usuario puede eliminar un proyecto existente.
Precondición	El usuario está registrado y el proyecto existe.
Entrada	NA
Salida	NA
Secuencia normal	1. Desde la página principal, el usuario selecciona el proyecto deseado y hace <i>click</i> en “Eliminar proyecto seleccionado”. 2. La aplicación muestra un diálogo para que el usuario confirme la eliminación. 3. El usuario hace <i>click</i> en “Confirmar” y la aplicación elimina el proyecto.
Postcondición	Proyecto eliminado y nodos del proyecto liberados.
Excepciones	NA
Comentarios	NA
Actores	Usuario registrado

Tabla A.16: Caso de Uso de Buscar proyecto

Requisito	Buscar proyecto.
Identificador	3.8
Prioridad	Alta
Descripción	El usuario puede buscar un proyecto concreto.
Precondición	El usuario está registrado y existe algún proyecto.
Entrada	Nombre del proyecto.
Salida	NA
Secuencia normal	1. Desde el menú principal, el usuario hace <i>click</i> en “Proyectos”. 2. La aplicación muestra la lista de proyectos. 3. El usuario introduce el nombre del proyecto deseado en la barra de búsqueda y hace <i>click</i> en “Buscar”. 4. La aplicación muestra los proyectos en orden de mayor a menor coincidencia.
Postcondición	NA
Excepciones	NA
Comentarios	NA
Actores	Usuario registrado

Tabla A.17: Caso de Uso de Actualización de modelos en proyectos

Requisito	Actualización de modelos en proyectos
Identificador	3.9
Prioridad	Alta
Descripción	El usuario puede iniciar una ronda de aprendizaje de todos los nodos de un proyecto.
Precondición	El usuario está registrado, el proyecto existe.
Entrada	Rondas a entrenar.
Salida	NA
Secuencia normal	1. Desde la página principal, el usuario selecciona el proyecto deseado y hace <i>click</i> en “Iniciar entrenamiento federado”. 2. La aplicación muestra un diálogo para que el usuario introduzca el número de rondas a entrenar. 3. El usuario introduce los datos y hace <i>click</i> en “Iniciar”. 4. La aplicación realiza el entrenamiento con los nodos durante las rondas seleccionadas. 5. Una vez terminado el aprendizaje, la aplicación muestra un diálogo con el texto: “Completado en Ts. Resultados guardados en el proyecto.”.
Postcondición	Modelos de los nodos actualizados con los nuevos valores y resultado de los entrenamientos añadidos a la base de datos.
Excepciones	■ En caso de no existir <i>dataset</i> en alguno de los nodos, se muestra un mensaje de error. ■ Si hay algún fallo en el proceso del entrenamiento, se muestra un mensaje de error. ■ Si el proyecto no tiene nodos asociados, se muestra un mensaje de error. ■ Si se deja en blanco el número de rondas, se muestra un mensaje de error.
Comentarios	T es el tiempo empleado en realizar el entrenamiento
Actores	Usuario registrado

Tabla A.18: Caso de Uso de Visualización de resultados y métricas del proyecto

Requisito	Visualización de resultados y métricas del proyecto
Identificador	3.10
Prioridad	Alta
Descripción	El usuario puede ver los resultados y métricas asociados a los entrenamientos de un proyecto.
Precondición	El usuario está registrado y el proyecto existe.
Entrada	NA
Salida	NA
Secuencia normal	1. Desde la página principal, el usuario selecciona el proyecto deseado y hace <i>click</i> en “Ver métricas”. 2. La aplicación muestra un diálogo en el que se dibujan gráficos relevantes de las métricas de los entrenamientos del proyecto.
Postcondición	NA
Excepciones	■ En caso de no haberse entrenado el modelo ninguna vez, se mostrará un mensaje de error.
Comentarios	NA
Actores	Usuario registrado

Manual de Instalación

Antes de comenzar, es importante revisar la instalación de Python. La versión recomendada para ejecutar el proyecto es **Python 3.12.10**.

A continuación, se detallan las instrucciones para instalar y ejecutar el proyecto:

1. Clonar el repositorio:

```
git clone git@github.com:serjim06/TFG-FederatedLearning.git
```

2. Navegar al repositorio y crear un entorno virtual:

```
cd TFG-FederatedLearning
python -m venv .venv
source .venv/bin/activate
```

3. Instalar las dependencias:

```
pip install -r requirements.txt
```

4. Configurar el .env con los datos del administrador (ejemplo en .env.example):

```
cp .env.example .env
```

5. Iniciar la base de datos:

```
python -m scripts.init_database [--nodes N] [--env RUTA]
```

6. Ejecutar el proyecto:

```
python -m src.main
```

Algunas notas:

- Es importante configurar el fichero `.env` antes de ejecutar el script de inicialización de la base de datos.
- En el script de inicialización de la base de datos, el argumento opcional `env` es la ruta del fichero **relativa** a la raíz del proyecto.

Manual de Usuario

A continuación, se presenta un manual de usuario, con el que se pretende ayudar al usuario a manejarse con la aplicación, explicando cómo acceder y utilizar cada una de las funcionalidades.

C.1. Autenticación

- **Inicio de sesión.** Al acceder a la aplicación, se muestra el formulario de inicio de sesión, en el cual se solicitan el nombre de usuario y la contraseña para poder acceder a la herramienta. Además, permite registrar nuevos usuarios y recuperar la contraseña en caso de pérdida.

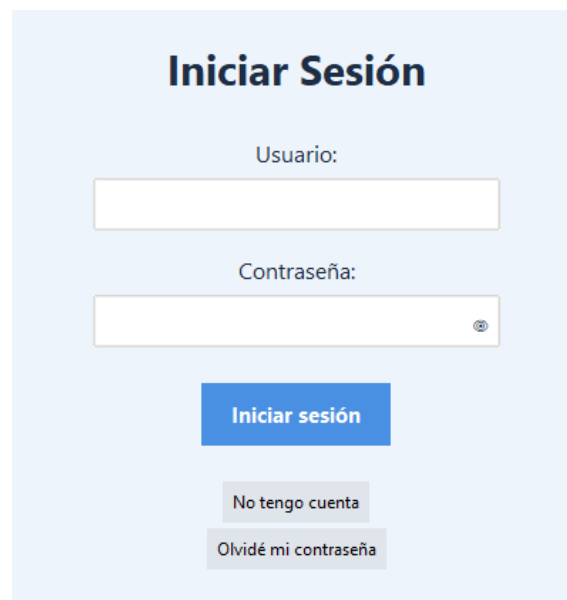


Figura C.1: Manual: Inicio de Sesión

- **Registro de un nuevo usuario.** Para registrar un nuevo usuario en la aplicación, hay que pulsar el botón “No tengo cuenta” en el formulario de inicio

de sesión. Esto abrirá la página de registro (figura C.2), en la que se definen el nombre de usuario, la contraseña y la frase de recuperación. Estas dos últimas hay que introducirlas dos veces, siendo exactamente iguales aquellas de la misma categoría. Para terminar el registro, hay que pulsar el botón “Registrar” para registrar al nuevo usuario en base de datos. Si los datos son correctos, se vuelve a la página de inicio de sesión.

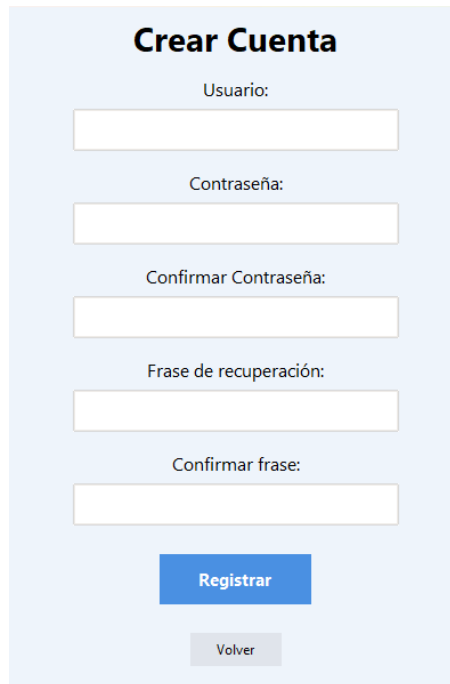
El formulario tiene un fondo azul claro. En la parte superior, el título "Crear Cuenta" está en negrita y color negro. Debajo, hay cinco campos de entrada de texto blancos con bordes grises, cada uno precedido por su etiqueta: "Usuario:", "Contraseña:", "Confirmar Contraseña:", "Frase de recuperación:" y "Confirmar frase:". Al final del formulario, hay dos botones: uno azul con el texto "Registrar" en blanco, y uno gris con el texto "Volver" en gris.

Figura C.2: Manual: Crear Cuenta

- **Recuperar contraseña.** En caso de olvidar la contraseña y querer cambiarla por una nueva, el usuario debe pulsar en el botón “Olvidé mi contraseña”. Tras esto, se mostrará una página en la que debe introducir el nombre del usuario del que quiere recuperar la contraseña.

El formulario tiene un fondo azul claro. En la parte superior, el título "Recuperar contraseña" está en negrita y color negro. Debajo, hay un campo de entrada de texto blanco con borde gris, precedido por la etiqueta "Introduce tu usuario:". Debajo del campo, hay dos botones: uno azul con el texto "Verificar usuario" en blanco, y uno gris con el texto "Volver" en gris.

Figura C.3: Manual: Introducir usuario para recuperar contraseña

Si el usuario existe en base de datos, se muestra la página en la que se debe introducir la frase de recuperación definida al registrar al usuario y una contraseña, repetida dos veces. En caso de coincidir todo y pulsar “Recuperar contraseña”, se pasará a la página de inicio de sesión, desde la que se podrá acceder con los nuevos datos.



Recuperar contraseña

Frase de recuperación:

Nueva contraseña:

Confirmar nueva contraseña:

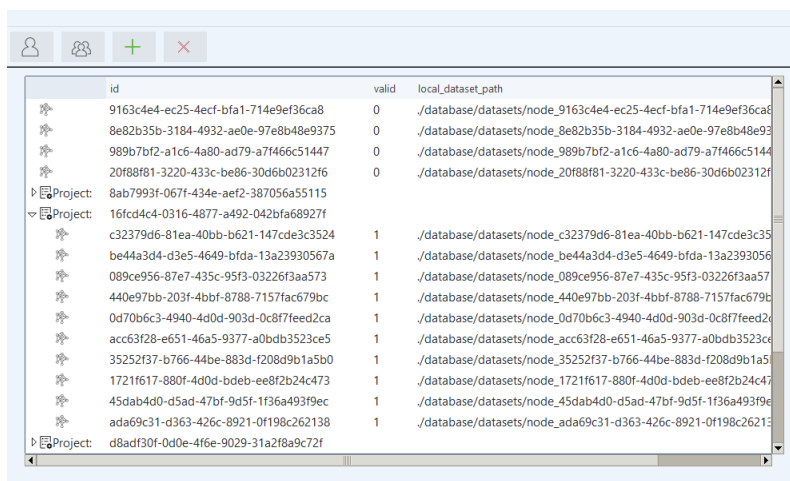
Recuperar contraseña

Volver

Figura C.4: Manual: Introducir frase de recuperación y contraseña nueva

C.2. Acciones del usuario administrador

Tras iniciar sesión, al administrador se le muestra la lista de nodos, distinguiendo de aquellos pertenecientes a los proyectos (está bajo el desplegable de un proyecto). Desde esta página, el usuario administrador puede realizar las siguientes acciones:



	id	valid	local_dataset_path
✖	9163c4e4-ec25-4ecf-bfa1-714e9ef36ca8	0	./database/datasets/node_9163c4e4-ec25-4ecf-bfa1-714e9ef36ca8
✖	8e82b35b-3184-4932-ae0e-97e8b48e9375	0	./database/datasets/node_8e82b35b-3184-4932-ae0e-97e8b48e9375
✖	989b7bf2-a1c6-4a80-ad79-a71466c51447	0	./database/datasets/node_989b7bf2-a1c6-4a80-ad79-a71466c51447
✖	20f88f81-3220-433c-be86-30d6b02312f6	0	./database/datasets/node_20f88f81-3220-433c-be86-30d6b02312f6
▶ Project:	8ab7993f-067f-434e-ae2f-387056a55115		
▼ Project:	16fcd4c4-0316-4877-a492-042bfa68927f		
✖	c32379d6-81ea-40bb-b621-147cde3c3524	1	./database/datasets/node_c32379d6-81ea-40bb-b621-147cde3c3524
✖	be44a3d4-d3e5-4649-bfda-13a23930567a	1	./database/datasets/node_be44a3d4-d3e5-4649-bfda-13a23930567a
✖	089ce956-87e7-435c-95f3-03226f3aa573	1	./database/datasets/node_089ce956-87e7-435c-95f3-03226f3aa573
✖	440e97bb-203f-4bbf-8788-7157fac679bc	1	./database/datasets/node_440e97bb-203f-4bbf-8788-7157fac679bc
✖	0d70b6c3-4940-4d0d-903d-0c8f7feed2ca	1	./database/datasets/node_0d70b6c3-4940-4d0d-903d-0c8f7feed2ca
✖	acc63f28-e651-46a5-9377-a0bdb3523ce5	1	./database/datasets/node_acc63f28-e651-46a5-9377-a0bdb3523ce5
✖	35252f37-b766-44be-883d-f208d9b1a5b0	1	./database/datasets/node_35252f37-b766-44be-883d-f208d9b1a5b0
✖	1721f617-880f-4d0d-bdeb-ee8f2b24c473	1	./database/datasets/node_1721f617-880f-4d0d-bdeb-ee8f2b24c473
✖	45dab4d0-d5ad-47bf-9d5f-1f36a493f9ec	1	./database/datasets/node_45dab4d0-d5ad-47bf-9d5f-1f36a493f9ec
✖	ada69c31-d363-426c-8921-0f198c262138	1	./database/datasets/node_ada69c31-d363-426c-8921-0f198c262138
▶ Project:	d8adf30f-0d0e-4f6e-9029-31a2f8a9c72f		

Figura C.5: Manual: Lista de nodos

- **Añadir nodo.** Para añadir un nuevo nodo, basta con pulsar el botón “Añadir un nuevo nodo a la base de datos” (tercer botón en la lista). Esto añade a la

lista de nodos uno nuevo con un ID aleatorio y un directorio para sus *datasets* definido.

- **Eliminar nodo.** Para eliminar un nodo de la base de datos, hay que seleccionar el nodo deseado de la lista y pulsar en el botón “Eliminar el nodo seleccionado de la base de datos” (cuarto botón en la lista). En este caso hay dos posibilidades:
 - **El nodo no pertenece a ningún proyecto:** En este caso, el nodo se elimina directamente de la base de datos.
 - **El nodo pertenece a un proyecto:** En este caso, se pedirá confirmación para eliminar el nodo, ya que ese nodo es utilizado por un usuario en un proyecto. Tras confirmar la eliminación, se elimina tanto el nodo de la base de datos como el directorio *datasets* asociado.
- **Ver lista de usuarios.** Para ver la lista de los usuarios registrados en la aplicación, basta con pulsar el botón “Gestión de usuarios” (segundo botón de la lista), que mostrará una lista con los usuarios, sus nombres, ID, proyectos, roles, fecha de creación, último inicio de sesión y último entrenamiento realizado.



Nombre: sergio User ID: 127548fc-6cf5-4f13-9307-f3a0aeeecd96 Amount of projects: 9 Role: user Creation date: - Last login: 2026-05-17 19:29:46 Last train: 2026-05-10 11:28:52
Nombre: test User ID: c9333a13-c5b6-43e4-b37-06e551b0c177 Amount of projects: 1 Role: user Creation date: 2026-05-05 15:22:33 Last login: 2026-05-05 15:22:39 Last train: -

Figura C.6: Manual: Lista de usuarios

- **Ver usuario registrado:** Desde la lista de usuarios, se puede acceder al perfil de un usuario, que muestra, además de los datos mostrados en la lista, un listado con sus proyectos, incluyendo nombre, fecha de creación, fecha de actualización e IDs de los nodos asociados.

User ID: 127548fc-6cf5-4f13-9307-f3a0aeeecd96

Nombre: sergio

Role: user

Creation date: -

Last login: 2026-05-17 19:29:46

Last train: 2026-05-10 11:28:52

Projects:

- Salary

Created at: 2026-05-05 15:37:24

Updated at: 2026-05-05 18:11:21

Nodes:

- 6844deff-9a00-43ab-9ad0-76a5809c9fb9

- Synth FedAvg

Created at: 2026-05-08 15:33:40

Updated at: 2026-05-08 15:38:42

Nodes:

- ada69c31-d363-426c-8921-0f198c262138

- 45dab4d0-d5ad-47bf-9d5f-1f36a493f9ec

- 1721f617-880f-4d0d-bdeb-ee8f2b24c473

Volver Eliminar Usuario

Figura C.7: Manual: Perfil de un usuario registrado

- **Eliminar usuario registrado:** Desde el perfil del usuario registrado (figura C.7), este puede ser eliminado pulsando el botón “Eliminar usuario”. Tras confirmar, se eliminarán todos sus proyectos, con sus directorios en *models* asociados. Además, se liberarán todos los nodos utilizados en sus proyectos, dejándolos libres para el uso de otro usuario.

C.3. Usuario registrado

Tras iniciar sesión, al usuario se le muestra su lista de proyectos. Desde esta página, el usuario administrador puede realizar las siguientes acciones:

	id	name	description	pending
Project	8ab7993f-067f-434e-aef2-387056a55115	Salary		False
Project	16fcd4c4-0316-4877-a492-042bfa68927f	Synth FedAv		False
Project	d8adf30f-0d0e-4f6e-9029-31a2f8a9c72f	Synth FedMe		False
Project	78f3714e-97c3-44f9-8762-3f15f59d4789	Synth SCAFF		False
Project	d9f48c02-c281-4354-8d20-03122325a418	Synth SSFed		False
Project	7653e605-7943-4678-9f23-edd1298565e3	Appliancd F		False
Project	b8affb41-760a-483c-be8e-9de723fc06ff	Appliance S		False
Project	f8f96397-59ac-4e90-8956-2791d57281d9	Appliance F		False
Project	c76a3fe1-855f-4b70-8046-90dbca5df587	Synth		False

Listo.

Figura C.8: Manual: Lista de proyectos

- **Crear un proyecto.** Para crear un nuevo proyecto, el usuario debe pulsar el botón “Crear un nuevo proyecto”. Esto muestra el formulario de creación de un nuevo proyecto. En este formulario deben introducirse los siguientes datos:
 - **Nombre:** Nombre del proyecto.
 - **Descripción:** Descripción del proyecto.
 - **Modelo:** Selección del modelo a utilizar. El fichero debe implementar la clase de la figura C.9.

```

6 class BaseModel(ABC):
7     """
8     User model contract: defines how to build/load the PyTorch network and which
9     columns/features the project uses. Training, evaluation, and prediction
10    are implemented in ``src.models.node``.
11    """
12
13    @abstractmethod
14    def load_model(self, model_path) -> nn.Module:
15        """Return the PyTorch module (architecture and, if applicable, weights loaded from ``model_path``)."""
16        pass
17
18    @abstractmethod
19    def get_features(self) -> dict:
20        """
21        Dictionary with at least ``input_features`` and ``output_features`` (lists of names).
22        Optionally ``metadata`` with ``type`` (``'regression'`` | ``'classification'``),
23        ``categorical_columns`` (input column names with categorical text to encode),
24        ``labels`` (classification), ``units``, etc., for reports and prediction formatting.
25        """
26        pass

```

Figura C.9: Manual: Modelo base

- **Tipo de tarea:** Selección de si el proyecto es de clasificación o regresión. Si en los metadatos del fichero se especifica el tipo de tarea, se marca automáticamente.
- **Función de pérdida:** Función de pérdida utilizada para entrenar al modelo, a elegir dependiendo del tipo de tarea.
- **Optimizador:** Optimizador a utilizar durante el entrenamiento. A elegir entre “Adam”, “SGD” y “RMSprop”.
- **Estrategia de agregación:** Estrategia seleccionada para agregar los pesos de los modelos. A elegir entre “FedAvg”, “FedMedian”, “SCAFFOLD” y “SSFed”.
- **Epochs:** Épocas de entrenamiento local.
- **Validation split:** Fracción del *dataset* utilizado para validación.
- **Batch size:** Tamaño de *batch* utilizado al preparar los datos. A mayor tamaño, mejor rendimiento, pero mayor capacidad de computación necesaria.
- **Fraction fit:** Fracción de los clientes mínima para hacer entrenamiento federado.
- **Fraction evaluate:** Fracción de los clientes mínima para hacer evaluación.
- **Learning rate:** Tasa de aprendizaje del servidor.

- **Nodos:** Selección de los nodos a utilizar en el proyecto.

Nombre:
Air Quality

Descripción:
Proyecto para predecir la calidad del aire

air_quality_mlp_model.py Seleccionar

Tipo de tarea:
classification

Función de pérdida:
categorical_crossentropy

► Parámetros

Optimizer:
Adam

Aggregation strategy:
fed_avg

Epochs:
3

Validation split:
0.20

Batch size:
32

Fraction fit:
0.80

Fraction evaluate:
0.50

Learning rate:
0.0100

Nodos:

- ☒ ada69c31-d363-426c-8921-0f198c262138
- ☒ 45dab4d0-d5ad-47bf-9d5f-1f36a49399ec
- ☒ 17211617-880f-4d0d-bdeb-ee9f2b24c473
- ☐ 35252f37-b766-44be-883d-f208d9b1a5b0
- ☐ accd3f28-e651-46a5-9377-a0bdb3523ce5

(a)
(b)

Figura C.10: Manual: Creación de un proyecto nuevo.

- **Eliminar un proyecto.** Desde el listado de proyectos, el usuario puede eliminar un proyecto seleccionándolo y pulsando el botón “Eliminar proyecto seleccionado”. Esto lo elimina de la base de datos, libera sus nodos y libera su espacio ocupado en el directorio *models*.
- **Buscar un proyecto.** Para buscar un proyecto, el usuario puede escribir en la barra de búsqueda en la parte superior derecha de la lista de proyectos. Conforme vaya escribiendo, la aplicación mostrará sugerencias en función de su parecido con lo escrito. Si se selecciona una opción, se escribe esta en la barra y se busca. La búsqueda, ya sea presionando “Enter”, “Buscar” o con las sugerencias, ordena la lista de proyectos en función del parecido con lo escrito en la barra de búsqueda.

👤 + ✖ ▶ ⚙ 🖨 📄 aplian Buscar			
id	name	description	pending
Project: f8196397-59ac-4e90-8956-2791d57281d9	Appliance F		False
Project: b8affb41-760a-483c-be8e-9de723fc06ff	Appliance S		False
Project: 8ab7993f-067f-434e-aef2-387056a55115	Salary		False
Project: 7653e605-7943-4678-9f23-edd1298565e3	Appliancd F		False
Project: c76a3fe1-855f-4b70-8046-90dbca5df587	Synth		False
Project: 16fcd4c4-0316-4877-a492-042bfa68927f	Synth FedAv		False
Project: d8adf30f-0d0e-4f6e-9029-31a2f8a9c72f	Synth FedMx		False
Project: d9f48c02-c281-4354-8d20-03122325a418	Synth SSFed		False
Project: 78f3714e-97c3-44f9-8762-3f15f59d4789	Synth SCAFF		False

Listo.

Figura C.11: Manual: Buscar proyecto

- **Configurar un proyecto.** Para configurar un proyecto existente, debe pulsar en el proyecto a configurar y, posteriormente, en el botón “Configurar el

proyecto seleccionado”. Esto mostrará el formulario de configuración, en el que se pueden modificar los siguientes aspectos:

- **Nombre del proyecto.**
- **Descripción.**
- **Nodos.** Se puede asignar nuevos nodos y eliminar nodos ya existentes en el proyecto.

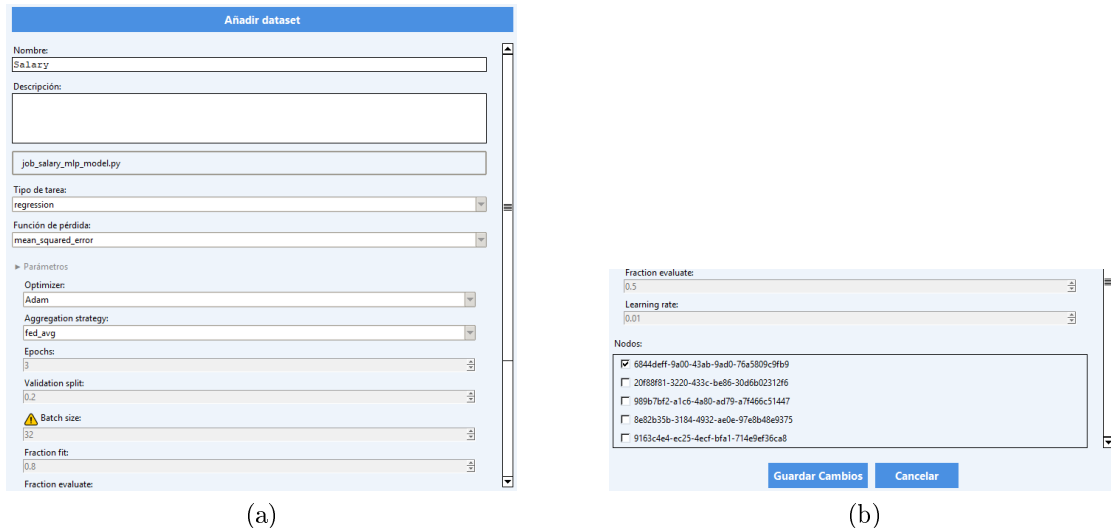


Figura C.12: Manual: Configuración de un proyecto.

- **Añadir un *dataset* a un nodo de un proyecto.** Para añadir un *dataset* a un proyecto, el usuario debe entrar en la configuración del proyecto deseado y pulsar en “Añadir dataset”. Esto muestra un diálogo en el que se puede elegir el nodo en que guardar el *dataset* y seleccionar el fichero *csv* del *dataset* a añadir. Este *dataset* debe cumplir una serie de condiciones:

- No debe tener líneas vacías.
- No debe tener huecos vacíos (e.g. 1.2, 3.4, 3.2, ...).
- Las columnas deben estar en el orden especificado en el modelo.

Si cumple estas condiciones, se añade el *dataset* al final del último si pertenecen a la misma ronda o, en otro caso, se añade al final de la copia del último *dataset*. Esto se hace con el objetivo de registrar los cambios en los datos y su influencia en los resultados.

Selecciona el nodo y un archivo CSV para añadir sus filas al dataset de la ronda actual. Las columnas deben coincidir con las características del proyecto (cabecera opcional); no se permiten filas vacías ni celdas vacías.

Nodo:

Archivo CSV:

Figura C.13: Manual: Añadir un dataset.

- **Entrenar una red.** El usuario puede entrenar la red de un proyecto pulsando el botón “Iniciar entrenamiento federado”. Esto muestra un diálogo en el que seleccionar el número de rondas a entrenar.

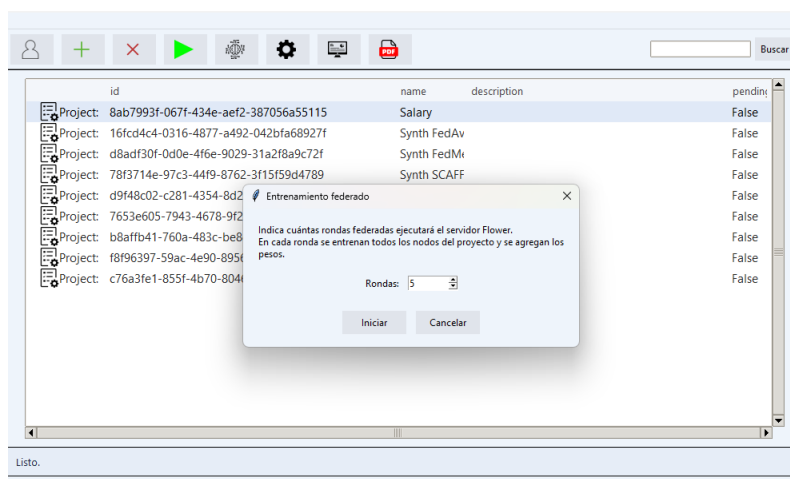


Figura C.14: Manual: Pantalla de preparación de entrenamiento.

Al pulsar en “Iniciar”, la red empieza a entrenar durante las rondas seleccionadas, mostrando el progreso en la parte inferior de la lista de proyectos. Tras entrenar y almacenar los resultados, se muestra un diálogo informando del éxito del mismo.

- **Realizar una predicción.** Para realizar una predicción en un nodo de un proyecto, el usuario debe seleccionar el proyecto deseado y pulsar en “Realizar predicción para el proyecto seleccionado”. Tras esto, se muestra un diálogo en el que se selecciona el nodo en el que realizar la predicción y se introducen los parámetros de entrada, en el mismo formato que en los *datasets* (e.g. 1,2,3,4,5,...). Tras introducir los datos y pulsar en “Predecir”, se realiza la predicción, que se guarda relacionada al nodo seleccionado en “Confirmaciones pendientes”.

Introduce los valores de entrada en el mismo orden que las columnas del proyecto, separados por comas (números decimales con punto).

Columnas: job_title, education_level, industry, company_size, location, remote_work, experience_years, skills_count, certifications

Nodo:

Figura C.15: Manual: Realizar una predicción.

- **Confirmar una predicción.** Tras realizar una predicción, esta se guarda en las confirmaciones pendientes del proyecto, que se refleja en la lista de proyectos.

	id	name	description	pending
Project:	8ab7993f-067f-434e-aef2-387056a55115	Salary		False
Project:	16fcd4c4-0316-4877-a492-042bfa68927f	Synth FedAv		True
Project:	d8adf30f-0d0e-4f6e-9029-31a2f8a9c72f	Synth FedMx		False
Project:	78f3714e-97c3-44f9-8762-3f15f59d4789	Synth SCAFF		False
Project:	d9f48c02-c281-4354-8d20-03122325a418	Synth SSFed		False
Project:	7653e605-7943-4678-9f23-edd1298565e3	Appliancd F		False
Project:	b8affb41-760a-483c-be8e-9de723fc06ff	Appliance S		False
Project:	f8f96397-59ac-4e90-8956-2791d57281d9	Appliance F		False
Project:	c76a3fe1-855f-4b70-8046-90dbca5df587	Synth		False

Listo.

Figura C.16: Manual: Lista de proyectos con confirmaciones pendientes.

El usuario puede confirmar estas predicciones pendientes entrando en la configuración del proyecto y pulsando en “Confirmar Resultados Pendientes” (figura C.17). Esto muestra un diálogo en el que aparecen los parámetros de entrada y salida. El usuario puede seleccionar “Correcto” si el resultado es correcto o “Incorrecto” en otro caso. En este último caso, además, el usuario debe corregir el resultado. Ya sea correcto o corregido, la versión aprobada por el usuario se guarda en los *datasets* del nodo relacionado de forma análoga a como se hace en “Añadir un *dataset* a un nodo de un proyecto”.

Confirmar Resultados Pendientes

Añadir dataset

Nombre:

Synth FedAvg

Descripción:

synthetic_leaf_mlp_model.py

Tipo de tarea:

classification

Función de pérdida:

categorical_crossentropy

▶ Parámetros

Optimizer:

Adam

Aggregation strategy:

fed_avg

Epochs:

9

Validation split:

0.2

⚠

Batch size:

128

Fraction fit:

Guardar Cambios

Cancelar

Figura C.17: Manual: Pantalla de configuración de un proyecto con confirmaciones pendientes.

re_56	Feature_57	Feature_58	Feature_59	Target	Acciones	
41056972	1.2289700639645136	0.2996967955829009	1.902600447995471	1	Correcto	Incorrecto

Volver

Figura C.18: Manual: Confirmar predicción.

- **Ver métricas de un proyecto.** El usuario puede ver las métricas de los entrenamientos realizados en un proyecto. Para esto tiene que seleccionar el

proyecto deseado y pulsar en “Ver las métricas del proyecto seleccionado”. Esto muestra un diálogo con la configuración del proyecto y algunas métricas de los entrenamientos. Tanto si es clasificación como regresión, se muestra el tiempo por ronda, los cambios en los *datasets* por ronda y un gráfico con la relación entre volumen de datos, participación de nodos y pérdida por ronda. Si es clasificación, además, se muestra una gráfica con pérdida y precisión por ronda. Si es regresión, se muestra una gráfica con la pérdida y puntuación R^2 por ronda y un gráfico de dispersión.

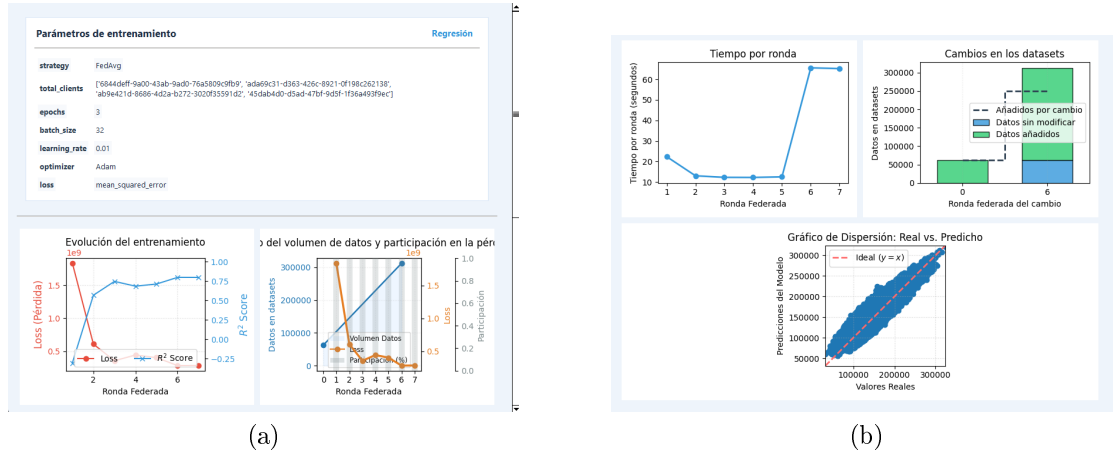


Figura C.19: Manual: Métricas de un proyecto.

- **Descargar informe de un proyecto.** El usuario puede descargar un informe en formato *PDF* con el resultado de los distintos entrenamientos realizados en un proyecto. Para esto, el usuario debe seleccionar el proyecto deseado y pulsar en “Descargar el reporte del proyecto seleccionado”. Después, el usuario debe seleccionar la ruta en la que guardar el informe. Tras esto, se crea y se guarda el informe en la ruta deseada.

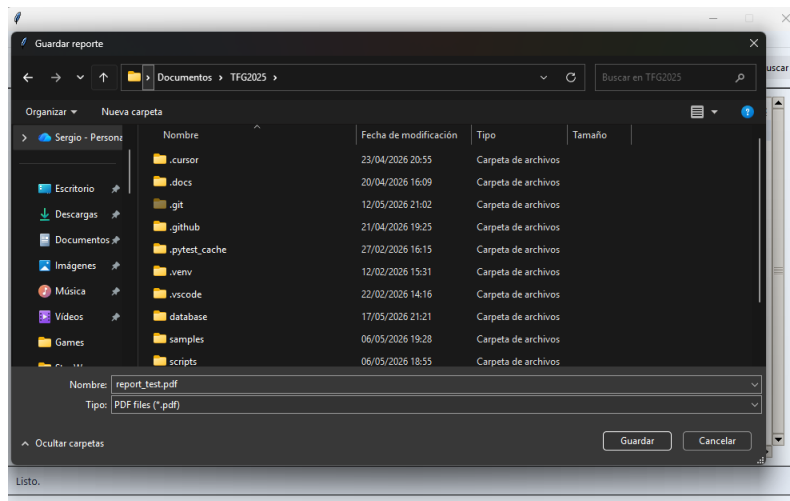


Figura C.20: Manual: Guardar un informe.

C.4. Gestión de cuenta

Para acceder a la información de su cuenta, el usuario debe pulsar el botón “Ver cuenta” desde la pantalla principal. Desde esta página, además de la visualización de la información de la cuenta, el usuario puede realizar las siguientes acciones:

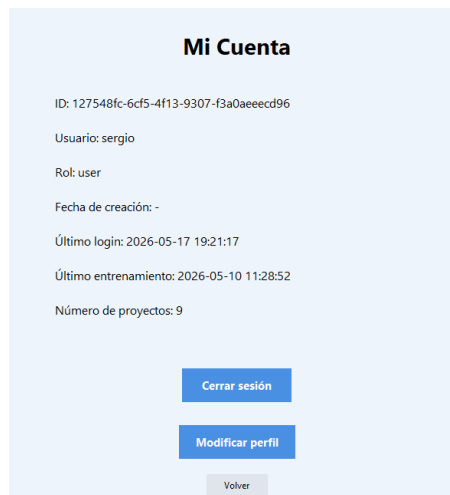


Figura C.21: Manual: Ver perfil.

- **Cerrar sesión.** Desde la pantalla del perfil, el usuario puede cerrar la sesión, pulsando el botón “Cerrar sesión”. Esto libera la sesión y muestra el formulario de inicio de sesión.
- **Modificar cuenta.** Desde la pantalla del perfil, el usuario puede modificar algunos de sus datos pulsando el botón “Modificar perfil”. Tras esto, se muestra un formulario en el cual se pueden modificar los siguientes campos:
 - **Nombre de usuario.**
 - **Contraseña.** Es necesario confirmarla repitiéndola exactamente en el campo “Confirmar Contraseña”.
 - **Frase de recuperación.** Es necesario confirmarla repitiéndola exactamente en el campo “Confirmar frase”.

Tras pulsar en “Confirmar cambios”, se modifican únicamente los campos que hayan sido modificados. Es decir, si se rellena únicamente los campos de contraseña y el resto se dejan igual, solamente se modifica la contraseña del usuario.



Modificar Perfil

Usuario:

Contraseña:

Confirmar Contraseña:

Nueva frase recuperación:

Confirmar frase:

[Confirmar cambios](#)

[Volver](#)

Figura C.22: Manual: Modificar perfil.

Apéndice **D**

Ejemplos de informe

A continuación, se muestra un ejemplo de informe de regresión y otro de clasificación:

D.1. Informe de regresión

Reporte de resultados del proyecto Appliance Fedavg

Información general

Fecha de generación: 2026-05-10 12:55:36

ID del proyecto: f8f96397-59ac-4e90-8956-2791d57281d9

Descripción del proyecto:

Tipo de proyecto: regression

Número de entrenamientos: 1

Número de rondas: 6

Configuración:

- Estrategia de agregación: FedAvg
- Número de épocas: 9
- Tamaño del lote: 128
- Tasa de aprendizaje: 0.0015
- Optimizador: Adam
- Función de pérdida: mean_squared_error

Entrenamientos

Entrenamiento 1

ID del entrenamiento: fed_558f8a10-6450-4234-b9ba-2186468a3393

Estrategia de agregación: FedAvg

Número total de nodos: 7

Nodos

- Nodo 0972d3cf-0371-44bd-ac23-c3a5884a48d1
- Nodo 0ac4d7cb-5942-4fb4-af7b-84e015d1af79
- Nodo 089a0aaf-2470-47c8-96b3-876431a2800a
- Nodo 4e63bb9d-bc84-4b99-ab19-9cc1b667d30f

- Nodo e439d3c4-3174-4fd0-bbc0-0cdef5253766
- Nodo cc7a455f-12c6-40cb-bf10-4357ce5bfa65
- Nodo e3bc1539-3a61-4430-b980-5509dfd245c7

Resultados por ronda

Ronda 1

- Pérdida global: 0.008938051428996686
- Puntuación R^2 : 0.03368765305917334

Ronda 2

- Pérdida global: 0.00785843805346566
- Puntuación R^2 : 0.11542410796489798

Ronda 3

- Pérdida global: 0.008599272710081218
- Puntuación R^2 : 0.13291076775833638

Ronda 4

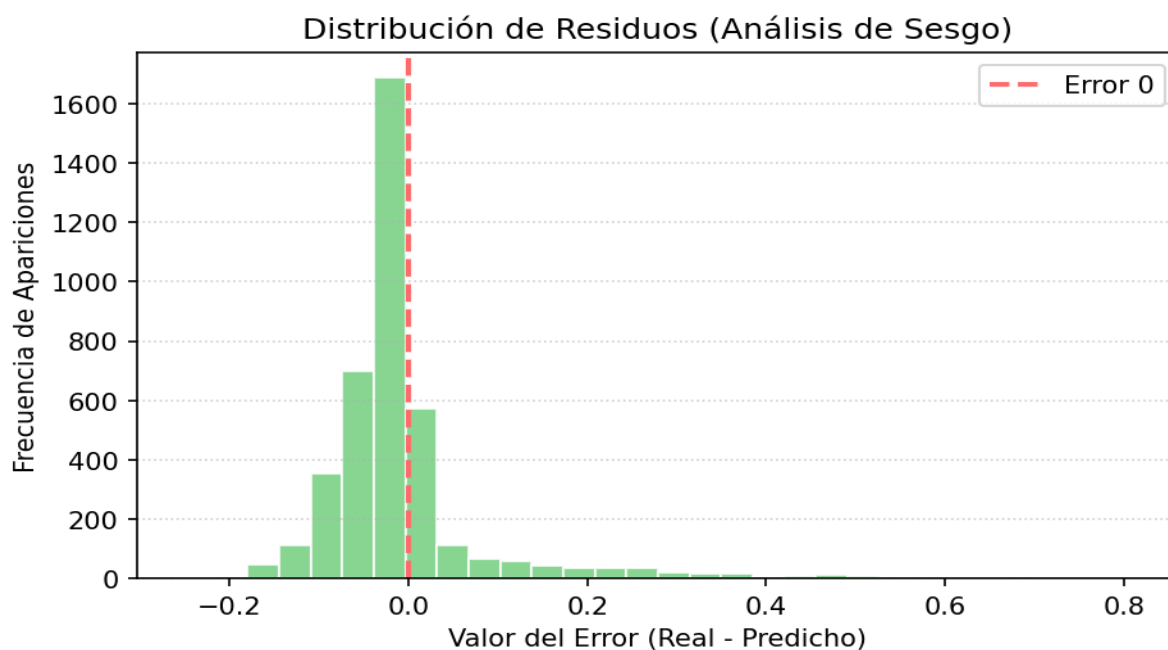
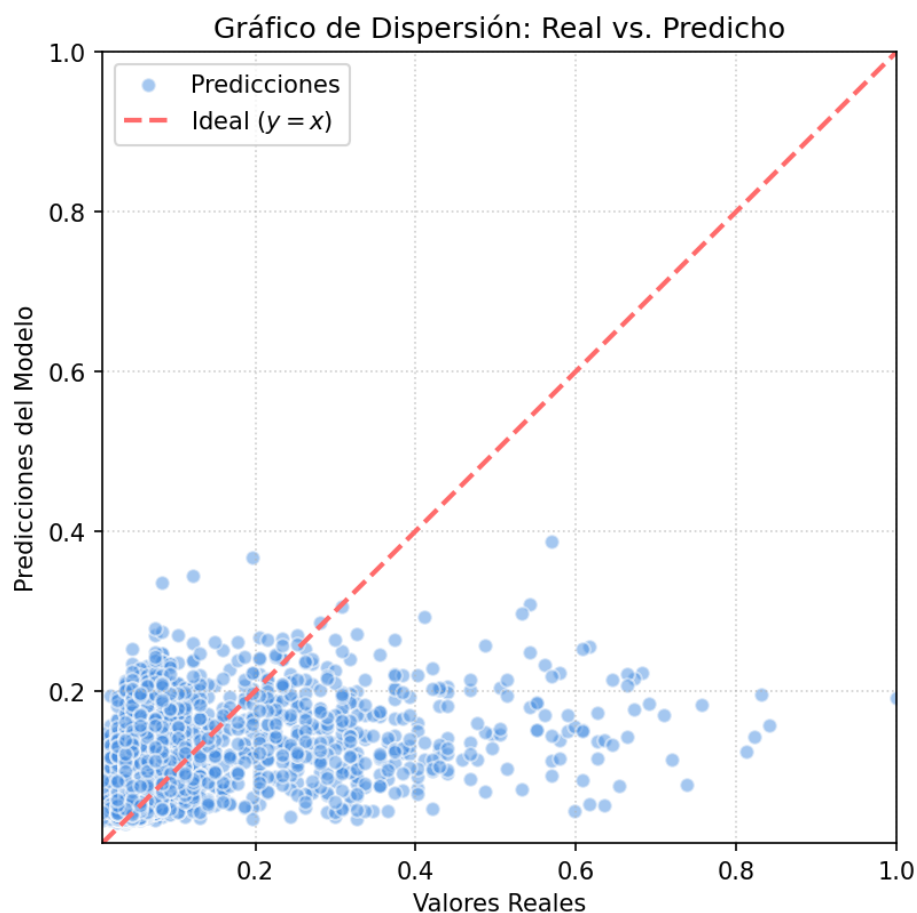
- Pérdida global: 0.0083078753903502
- Puntuación R^2 : 0.14166765094980707

Ronda 5

- Pérdida global: 0.008981536933028545
- Puntuación R^2 : 0.1702161379088689
- Error absoluto medio: 0.05461238326670385

Conclusiones del entrenamiento

Tiempo total del entrenamiento: 23.161232800000107 segundos



D.2. Informe de clasificación

Reporte de resultados del proyecto Synth

Información general

Fecha de generación: 2026-05-10 13:31:39

ID del proyecto: c76a3fe1-855f-4b70-8046-90dbca5df587

Descripción del proyecto:

Tipo de proyecto: classification

Número de entrenamientos: 1

Número de rondas: 6

Configuración:

- Estrategia de agregación: FedAvg
- Número de épocas: 9
- Tamaño del lote: 128
- Tasa de aprendizaje: 0.0015
- Optimizador: Adam
- Función de pérdida: categorical_crossentropy
- Clases: ['0', '1', '2', '3', '4']

Entrenamientos

Entrenamiento 1

ID del entrenamiento: fed_5917a197-b44c-4ca2-add6-0933725475ba

Estrategia de agregación: FedAvg

Número total de nodos: 5

Nodos

- Nodo fa4610b9-26dc-4a93-895f-db760a011292
- Nodo 7ae7ec70-ccc2-4757-bffd-5f0ee13d7605
- Nodo 3da58d67-5e1d-4e60-a79c-2d26b9304cd1
- Nodo eb63b1e6-ada9-47cc-be76-cb1e5f9c0130

- Nodo 31c198a7-6267-4f95-9da7-c4c5440573b0

Resultados por ronda

Ronda 1

- Pérdida global: 1.2227132409268624
- Precisión global: 0.6089965401017542

Ronda 2

- Pérdida global: 0.8938189968067212
- Precisión global: 0.5605536332953348

Ronda 3

- Pérdida global: 0.8090053790770112
- Precisión global: 0.6107266440030483

Ronda 4

- Pérdida global: 0.7320380575962935
- Precisión global: 0.7110726643276358

Ronda 5

- Pérdida global: 0.6907505745286102
- Precisión global: 0.7231833863822955

Conclusiones del entrenamiento 1

Tiempo total del entrenamiento: 19.3501641999992 segundos

Precisión, recall y f1_score por nodo y por clase

Nodo	Clase	Precisión	Recall	F1_score
fa4610b9-26dc-4a93-895f-db760a011292	0	0	0	0
fa4610b9-26dc-4a93-895f-db760a011292	1	0.0	0	0
fa4610b9-26dc-4a93-895f-db760a011292	2	0	0	0
fa4610b9-26dc-4a93-895f-db760a011292	3	1.0	0.910828025477707	0.9533333333333334
fa4610b9-26dc-4a93-895f-db760a011292	4	0	0	0
7ae7ec70-ccc2-4757-bffd-5f0ee13d7605	0	0	0	0
7ae7ec70-ccc2-4757-bffd-5f0ee13d7605	1	0	0	0
7ae7ec70-ccc2-4757-bffd-5f0ee13d7605	2	0	0	0
7ae7ec70-ccc2-4757-bffd-5f0ee13d7605	3	1.0	0.4166666666666667	0.5882352941176471
7ae7ec70-ccc2-4757-bffd-5f0ee13d7605	4	0.17647058823529413	1.0	0.3
3da58d67-5e1d-4e60-a79c-2d26b9304cd1	0	0.0	0	0
3da58d67-5e1d-4e60-a79c-2d26b9304cd1	1	0	0	0

3da58d67-5e1d-4e60-a79c-2d26b9304cd1	2	1.0	0.0547945205479452	0.10389610389610389
3da58d67-5e1d-4e60-a79c-2d26b9304cd1	3	0	0	0
3da58d67-5e1d-4e60-a79c-2d26b9304cd1	4	0	0	0
eb63b1e6-ada9-47cc-be76-cb1e5f9c0130	0	0.0	0	0
eb63b1e6-ada9-47cc-be76-cb1e5f9c0130	1	0	0	0
eb63b1e6-ada9-47cc-be76-cb1e5f9c0130	2	1.0	0.9871794871794872	0.9935483870967742
eb63b1e6-ada9-47cc-be76-cb1e5f9c0130	3	0	0	0
eb63b1e6-ada9-47cc-be76-cb1e5f9c0130	4	0	0	0
31c198a7-6267-4f95-9da7-c4c5440573b0	0	0	0	0
31c198a7-6267-4f95-9da7-c4c5440573b0	1	0	0	0
31c198a7-6267-4f95-9da7-c4c5440573b0	2	0	0	0
31c198a7-6267-4f95-9da7-c4c5440573b0	3	0	0	0
31c198a7-6267-4f95-9da7-c4c5440573b0	4	1.0	1.0	1.0

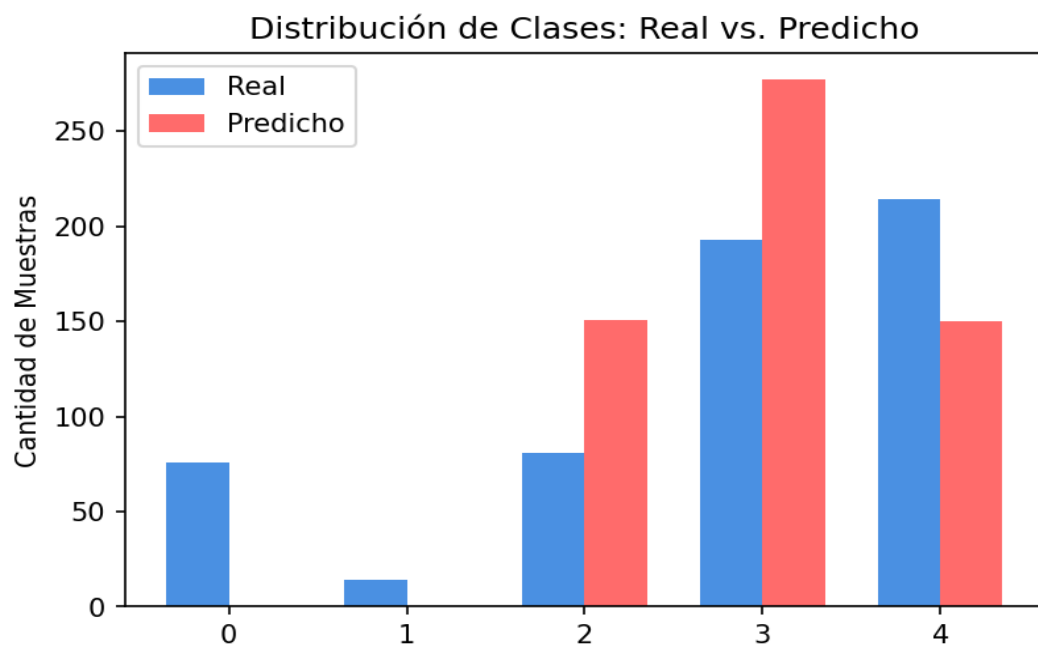
Matriz de confusión

Clase	0	1	2	3	4
0	0	0	70	0	6
1	0	0	0	14	0
2	0	0	81	0	0
3	0	0	0	193	0
4	0	0	0	70	144

Distribución de clases por nodo

Nodo	0	1	2	3	4
fa4610b9-26dc-4a93-895f-db760a011292	0	14	0	143	0
7ae7ec70-ccc2-4757-bffd-5f0ee13d7605	0	0	0	50	85
3da58d67-5e1d-4e60-a79c-2d26b9304cd1	75	0	4	0	0
eb63b1e6-ada9-47cc-be76-cb1e5f9c0130	1	0	77	0	0
31c198a7-6267-4f95-9da7-c4c5440573b0	0	0	0	0	129

Comparativa de frecuencias: Real vs Predicho:



*Si es bueno vivir, todavía es mejor soñar, y
lo mejor de todo, despertar.*

Antonio Machado

